

ROKE INDUSTRIES

# Documentacion Tecnica Completa

Hosting Platform — Backend & Frontend

15 de mayo de 2026



API Reference · Arquitectura Tecnica · Funcionalidades del Sistema

PARTE I

# Backend

Laravel 10 · PHP 8.2 · MySQL · Redis · Reverb

## ROKE Industries — Hosting Platform Backend

### Descripción general

API REST construida con **Laravel 10** que alimenta el portal de clientes y el panel de administración de ROKE Industries. Gestiona usuarios, servicios de hosting, servidores de juego, VPS, facturación electrónica (CFDI 4.0), soporte y cotizaciones.

### Stack tecnológico

| Componente              | Tecnología                              |
|-------------------------|-----------------------------------------|
| Framework               | Laravel 10                              |
| Lenguaje                | PHP 8.2+                                |
| Base de datos           | MySQL / MariaDB                         |
| Caché / Colas           | Redis                                   |
| Autenticación           | Laravel Sanctum (cookies HttpOnly)      |
| WebSockets              | Laravel Reverb + Broadcasting           |
| Correo                  | SendGrid                                |
| Pagos                   | Stripe                                  |
| Facturación electrónica | Facturama (CFDI 4.0 SAT México)         |
| Game servers            | Pterodactyl Panel API + Wings WebSocket |
| Hosting compartido      | HestiaCP API                            |
| VPS                     | Proxmox API                             |
| DNS / Dominios          | Namecheap API + Cloudflare API          |

## Estructura de directorios

```
app/
├─ Console/           # Comandos artisan
├─ Events/            # Eventos de broadcasting
├─ Exceptions/        # Manejo global de excepciones
├─ Http/
│   └─ Controllers/
│       ├── Admin/     # Controladores del panel admin
│       ├── Auth/      # Autenticación, 2FA, OAuth
│       ├── Client/    # Controladores del portal cliente
│       ├── Api/       # Endpoints públicos
│       └─ Common/     # Compartidos (webhooks, etc.)
└─ Middleware/        # Middlewares personalizados
├─ Models/            # Modelos Eloquent
├─ Notifications/     # Notificaciones (DB + Email)
├─ Providers/         # Service providers
└─ Services/          # Lógica de negocio / integraciones externas

routes/
├─ api.php            # Rutas públicas + hosting
├─ auth.php           # Autenticación
├─ client.php         # Portal cliente (auth requerida)
└─ admin.php          # Panel admin (auth + rol admin)
```

## Documentación detallada

- [API Reference](#) — Listado completo de endpoints, parámetros y respuestas
- [Arquitectura técnica](#) — Diseño del sistema, integraciones, flujo de autenticación, modelos de datos

## Configuración rápida

```
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan reverb:start      # WebSockets
php artisan queue:work redis  # Colas
```

## Variables de entorno principales

| Variable                                                                                  | Descripción                    |
|-------------------------------------------------------------------------------------------|--------------------------------|
| <code>APP_URL</code>                                                                      | URL base de la aplicación      |
| <code>DB_*</code>                                                                         | Conexión MySQL                 |
| <code>REDIS_*</code>                                                                      | Conexión Redis                 |
| <code>STRIPE_KEY</code> / <code>STRIPE_SECRET</code>                                      | Credenciales Stripe            |
| <code>PTERODACTYL_API_URL</code> / <code>PTERODACTYL_API_KEY</code>                       | API de Pterodactyl Panel       |
| <code>HESTIA_HOST</code> / <code>HESTIA_TOKEN</code>                                      | API de HestiaCP                |
| <code>PROXMOX_HOST</code> / <code>PROXMOX_USER</code> / <code>PROXMOX_PASSWORD</code>     | API de Proxmox                 |
| <code>NAMECHEAP_API_KEY</code> / <code>NAMECHEAP_USERNAME</code>                          | Namecheap API                  |
| <code>CLOUDFLARE_TOKEN</code> / <code>CLOUDFLARE_ZONE_ID</code>                           | Cloudflare API                 |
| <code>FACTURAMA_USER</code> / <code>FACTURAMA_PASSWORD</code>                             | Facturama (CFDI)               |
| <code>SENDGRID_API_KEY</code>                                                             | SendGrid correo transaccional  |
| <code>REVERB_APP_ID</code> / <code>REVERB_APP_KEY</code> / <code>REVERB_APP_SECRET</code> | Laravel Reverb WebSockets      |
| <code>GOOGLE_CLIENT_ID</code> / <code>GOOGLE_CLIENT_SECRET</code>                         | Google OAuth                   |
| <code>SANCTUM_STATEFUL_DOMAINS</code>                                                     | Dominios permitidos Sanctum    |
| <code>SESSION_DOMAIN</code>                                                               | Dominio de la cookie de sesión |

## Arquitectura técnica — Hosting Platform Backend

### Visión general

El backend sigue una arquitectura **MVC clásica de Laravel** con capas de servicio para las integraciones externas. El sistema orquesta múltiples proveedores de infraestructura (Pterodactyl, HestiaCP, Proxmox, Namecheap, Cloudflare) bajo una API REST unificada, con autenticación stateful via cookies Sanctum y notificaciones en tiempo real por WebSockets (Reverb).

## Flujo de autenticación

### Login estándar



### Verificación 2FA

```

POST /auth/2fa/verify
{ code: "123456", temp_token: "xxx" }
→ Valida TOTP con Google Authenticator / Authy
→ Si válido: emite cookie auth_token completa
  
```

### Google OAuth

```

Frontend obtiene id_token de Google
→ POST /auth/google/callback { credential: "id_token" }
→ Backend verifica con Google API
→ Si usuario nuevo: retorna { setup_token, requires_setup: true }
→ Frontend redirige a /auth/complete-profile
→ POST /auth/complete-profile { setup_token, username }
→ Cookie emitida
  
```

### Middleware de autenticación

| Middleware                         | Función                                                                                           |
|------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>auth:sanctum</code>          | Valida la cookie <code>auth_token</code> en cada request                                          |
| <code>InjectTokenFromCookie</code> | Inyecta el token de la cookie en el header <code>Authorization</code> para que Sanctum lo procese |
| <code>session.timeout</code>       | Cierra sesión automáticamente tras N minutos de inactividad (configurable en <code>.env</code> )  |
| <code>admin</code>                 | Verifica que el usuario tenga rol <code>admin</code> o <code>support</code>                       |
| <code>TrackUserSession</code>      | Registra IP, user-agent y timestamp en <code>UserSession</code>                                   |
| <code>throttle:10,1</code>         | Rate limit de 10 req/min para registro y OAuth                                                    |
| <code>throttle:5,1</code>          | Rate limit de 5 req/min para login y 2FA                                                          |

## Modelos de datos

### User

| Campo                           | Tipo       | Descripción                                                         |
|---------------------------------|------------|---------------------------------------------------------------------|
| <code>uuid</code>               | UUID       | Identificador público                                               |
| <code>name</code>               | string     | Nombre completo                                                     |
| <code>email</code>              | string     | Email único                                                         |
| <code>username</code>           | string     | Username único (requerido para Google OAuth completado)             |
| <code>password</code>           | hashed     | Contraseña                                                          |
| <code>role</code>               | enum       | <code>client</code> , <code>admin</code> , <code>support</code>     |
| <code>status</code>             | enum       | <code>active</code> , <code>suspended</code> , <code>pending</code> |
| <code>google_id</code>          | string?    | ID de cuenta Google                                                 |
| <code>avatar</code>             | string?    | URL del avatar                                                      |
| <code>two_factor_secret</code>  | encrypted? | Secreto TOTP                                                        |
| <code>two_factor_enabled</code> | bool       | Estado 2FA                                                          |

Relaciones: `services`, `invoices`, `tickets`, `paymentMethods`, `sessions`, `fiscalProfiles`, `notifications`

### Service

| Campo                       | Tipo       | Descripción                                                                                                     |
|-----------------------------|------------|-----------------------------------------------------------------------------------------------------------------|
| <code>uuid</code>           | UUID       | Identificador público                                                                                           |
| <code>user_id</code>        | FK         | Propietario                                                                                                     |
| <code>plan_id</code>        | FK         | Plan de servicio                                                                                                |
| <code>status</code>         | enum       | <code>active</code> , <code>suspended</code> , <code>cancelled</code> , <code>pending</code>                    |
| <code>category</code>       | string     | Tipo de servicio ( <code>game_server</code> , <code>hosting</code> , <code>vps</code> , <code>database</code> ) |
| <code>name</code>           | string     | Nombre descriptivo                                                                                              |
| <code>configuration</code>  | JSON       | Configuración técnica (IP, puerto, egg_id, etc.)                                                                |
| <code>billing_cycle</code>  | string     | Ciclo de facturación                                                                                            |
| <code>next_due_date</code>  | date       | Próxima fecha de cobro                                                                                          |
| <code>provisioned_at</code> | timestamp? | Fecha de aprovisionamiento                                                                                      |

Relaciones: `user`, `plan`, `invoices`, `tickets`, `backups`

## Invoice

| Campo                                 | Tipo       | Descripción                                                                              |
|---------------------------------------|------------|------------------------------------------------------------------------------------------|
| <code>uuid</code>                     | UUID       | Identificador público                                                                    |
| <code>user_id</code>                  | FK         | Cliente                                                                                  |
| <code>service_id</code>               | FK?        | Servicio relacionado                                                                     |
| <code>status</code>                   | enum       | <code>pending</code> , <code>paid</code> , <code>cancelled</code> , <code>overdue</code> |
| <code>total</code>                    | decimal    | Monto total                                                                              |
| <code>currency</code>                 | string     | <code>MXN</code> / <code>USD</code>                                                      |
| <code>due_date</code>                 | date       | Fecha de vencimiento                                                                     |
| <code>paid_at</code>                  | timestamp? | Fecha de pago                                                                            |
| <code>stripe_payment_intent_id</code> | string?    | ID de PaymentIntent Stripe                                                               |
| <code>cfdi_uuid</code>                | string?    | UUID del CFDI SAT                                                                        |
| <code>cfdi_status</code>              | enum?      | Estado del timbrado CFDI                                                                 |

Relaciones: `items`, `user`, `service`, `receipt`, `transactions`

## Ticket

| Campo                    | Tipo   | Descripción                                                                                                              |
|--------------------------|--------|--------------------------------------------------------------------------------------------------------------------------|
| <code>uuid</code>        | UUID   | Identificador público                                                                                                    |
| <code>user_id</code>     | FK     | Cliente                                                                                                                  |
| <code>service_id</code>  | FK?    | Servicio relacionado                                                                                                     |
| <code>agent_id</code>    | FK?    | Agente asignado                                                                                                          |
| <code>category_id</code> | FK?    | Categoría                                                                                                                |
| <code>status</code>      | enum   | <code>open</code> , <code>in_progress</code> , <code>waiting_client</code> , <code>resolved</code> , <code>closed</code> |
| <code>priority</code>    | enum   | <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code>                                       |
| <code>subject</code>     | string | Asunto                                                                                                                   |
| <code>body</code>        | text   | Contenido inicial                                                                                                        |

Relaciones: `user`, `agent`, `replies`, `service`

## ServicePlan

| Campo                    | Tipo   | Descripción                                 |
|--------------------------|--------|---------------------------------------------|
| <code>uuid</code>        | UUID   | Identificador público                       |
| <code>name</code>        | string | Nombre del plan                             |
| <code>slug</code>        | string | Slug URL                                    |
| <code>category_id</code> | FK     | Categoría                                   |
| <code>status</code>      | enum   | <code>active</code> , <code>inactive</code> |
| <code>specs</code>       | JSON   | Especificaciones técnicas                   |
| <code>is_featured</code> | bool   | Plan destacado                              |

Relaciones: `category`, `pricings`, `features`, `addOns`, `services`

## Quotation

| Campo                                                | Tipo    | Descripción                                                                                                                         |
|------------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>uuid</code>                                    | UUID    | Identificador público                                                                                                               |
| <code>token</code>                                   | string  | Token de acceso público (URL pública)                                                                                               |
| <code>status</code>                                  | enum    | <code>draft</code> , <code>sent</code> , <code>viewed</code> , <code>accepted</code> , <code>rejected</code> , <code>expired</code> |
| <code>client_name</code> / <code>client_email</code> | string  | Datos del cliente                                                                                                                   |
| <code>items</code>                                   | JSON    | Líneas de cotización                                                                                                                |
| <code>total</code>                                   | decimal | Total                                                                                                                               |
| <code>valid_until</code>                             | date    | Fecha de expiración                                                                                                                 |
| <code>version</code>                                 | int     | Número de versión (para revisiones)                                                                                                 |

## GameSoftwareVersion

| Campo                       | Tipo    | Descripción                                                    |
|-----------------------------|---------|----------------------------------------------------------------|
| <code>software</code>       | string  | Identificador (ej. <code>paper</code> , <code>vanilla</code> ) |
| <code>version</code>        | string  | Número de versión                                              |
| <code>download_url</code>   | string? | URL de descarga                                                |
| <code>is_enabled</code>     | bool    | Disponible para los clientes                                   |
| <code>is_recommended</code> | bool    | Versión recomendada                                            |



## PterodactylEgg

| Campo                       | Tipo    | Descripción                    |
|-----------------------------|---------|--------------------------------|
| <code>pterodactyl_id</code> | int     | ID en Pterodactyl Panel        |
| <code>nest_id</code>        | int     | ID del nido en Pterodactyl     |
| <code>name</code>           | string  | Nombre del egg                 |
| <code>display_name</code>   | string? | Nombre visible para el cliente |
| <code>description</code>    | text?   | Descripción                    |
| <code>is_active</code>      | bool    | Visible en el catálogo         |

## Integraciones externas

### Pterodactyl Panel

- **Propósito:** Aprovisionamiento, gestión y monitoreo de game servers.
- **Protocolo:** REST API + Wings WebSocket.
- **Autenticación:** API Key en header `Authorization: Bearer`.
- **Flujo de aprovisionamiento:**
  1. Admin crea servicio → `POST /admin/game-servers/{id}/provision`
  2. Backend llama a Pterodactyl API → crea servidor con egg, recursos y env vars
  3. Pterodactyl Wings provisiona el contenedor Docker
  4. Cliente obtiene credenciales WebSocket via `/game-server/websocket`
  5. Frontend conecta directamente a Wings para la consola en tiempo real

### HestiaCP

- **Propósito:** Hosting compartido (cuentas cPanel, bases de datos, emails, DNS).
- **Protocolo:** REST API.
- **Autenticación:** Token de API en header.
- **Operaciones:** Crear/suspender cuenta, gestionar bases de datos, emails y dominios.

### Proxmox

- **Propósito:** VPS (máquinas virtuales).
- **Protocolo:** REST API.
- **Autenticación:** Usuario + contraseña + ticket de sesión.
- **Operaciones:** Crear VM, snapshot, power control.

### Stripe

- **Propósito:** Procesamiento de pagos.
- **Integración frontend:** `@stripe/react-stripe-js` con `SetupIntent` para guardar tarjetas.
- **Integración backend:** Laravel Cashier + webhooks ( `POST /stripe/webhook` ).
- **Flujo de pago:**
  1. Frontend solicita `POST /payments/setup-intent` → Stripe `SetupIntent`
  2. Stripe.js captura datos de tarjeta de forma segura

3. Método guardado como `PaymentMethod` en la DB
4. `POST /payments/process` → cobra PaymentIntent en Stripe → crea `Transaction` + actualiza `Invoice`

### Facturama (CFDI 4.0 SAT México)

- **Propósito:** Emisión de facturas electrónicas conforme al SAT.
- **Flujo:**
  1. Pago exitoso → backend genera CFDI via Facturama API
  2. SAT timbra el comprobante → devuelve UUID + XML sellado
  3. Se guarda `cfdi_uuid` en `Invoice`
  4. Cliente puede descargar PDF/XML via `/invoices/{uuid}/pdf` y `/xml`
  5. Si falla el timbrado: admin puede reintentar via `/admin/cfdi/{id}/retry`
  6. Cancelación: `/admin/cfdi/{id}/cancel` → cancela ante el SAT

### Namecheap

- **Propósito:** Registro y gestión de dominios.
- **Operaciones:** Verificar disponibilidad, registrar, renovar, transferir.

### Cloudflare

- **Propósito:** DNS y protección para dominios de clientes.
- **Operaciones:** Crear/actualizar registros DNS, gestionar zonas.

### SendGrid

- **Propósito:** Correo transaccional.
- **Usos:** Verificación de email, reset de contraseña, recordatorios de facturas, notificaciones de soporte.

### Google OAuth

- **Propósito:** Login social.
- **Flujo:** Frontend obtiene `id_token` de Google → backend verifica con Google API → crea/asocia usuario.
- **Panel admin:** Valida adicionalmente que el usuario tenga rol `admin` o `support`.

## WebSockets (Laravel Reverb)

- **Servidor:** Laravel Reverb (alternativa open-source a Pusher).
- **Configuración:** `REVERB_APP_ID`, `REVERB_APP_KEY`, `REVERB_APP_SECRET`, `REVERB_HOST`, `REVERB_PORT`.
- **Autenticación de canales privados:** `POST /broadcasting/auth` valida la cookie Sanctum.
- **Canales:**
  - `private-user.{userId}` — notificaciones personales del cliente
  - `private-admin.{userId}` — notificaciones del panel admin
  - `private-chat.{roomId}` — mensajes de chat de soporte en tiempo real
  - Game server: el cliente conecta **directamente** a Wings (Pterodactyl), no pasa por Reverb

## Sistema de colas (Redis)

| Cola                       | Uso                                           |
|----------------------------|-----------------------------------------------|
| <code>default</code>       | Trabajos generales                            |
| <code>emails</code>        | Envío de correos via SendGrid                 |
| <code>notifications</code> | Notificaciones push a usuarios                |
| <code>cfdi</code>          | Timbrado y cancelación de CFDIs               |
| <code>provisioning</code>  | Aprovisionamiento de servidores (largo plazo) |

Comando para procesar: `php artisan queue:work redis --queue=emails,notifications,cfdi,provisioning,default`

## Arquitectura de módulos



## Flujo de aprovisionamiento de servicios

---

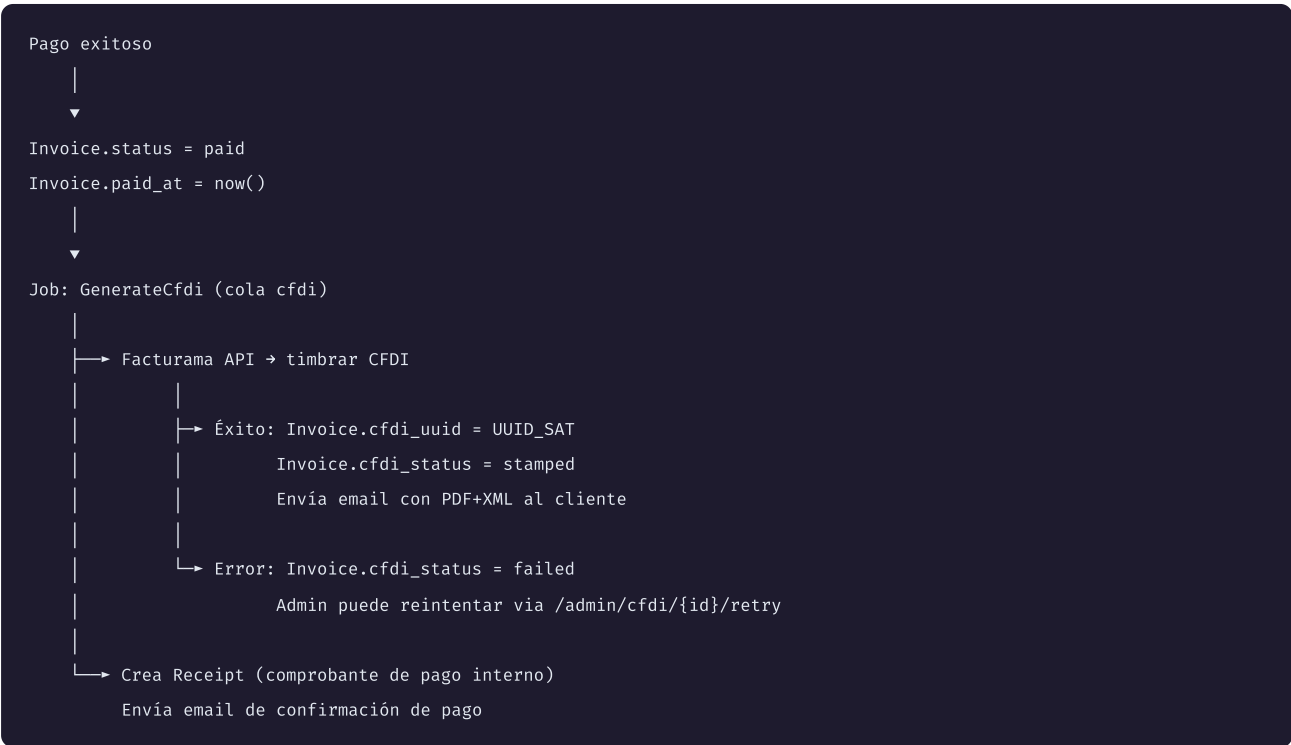
### Game Server

1. Cliente hace checkout → POST /services/contract  
{ plan\_id, configuration: { egg\_id, game, ... } }
2. Backend crea Service en DB (status: pending)
3. Admin confirma / pago procesado → POST /admin/game-servers/{id}/provision
4. PterodactylService:
  - Crea usuario en Pterodactyl (si no existe)
  - Crea servidor con egg\_id, resources (ram, disk, cpu), env vars
  - Pterodactyl retorna server\_id y credenciales
5. Backend actualiza Service:
  - configuration.pterodactyl\_id = server\_id
  - status = active
6. Cliente accede al servidor:
  - GET /services/{uuid}/game-server/websocket → obtiene token Wings
  - Frontend conecta WebSocket a Wings para consola en tiempo real

### Hosting

1. POST /services/contract → Service creado (status: pending)
2. Admin provisiona → HestiaService.createAccount(domain, username, password, package)
3. HestiaCP crea cuenta con bases de datos, emails, FTP
4. Service.status = active, configuration.hestia\_user = username

## Ciclo de vida de facturas y CFDI



## Seguridad

| Medida            | Implementación                                                         |
|-------------------|------------------------------------------------------------------------|
| Autenticación     | Cookie HttpOnly (no accesible por JS) + Sanctum                        |
| CSRF              | Token XSRF-TOKEN en cookie + header X-XSRF-TOKEN                       |
| 2FA               | TOTP (Google Authenticator / Authy)                                    |
| Rate limiting     | Throttle middleware (10 req/min registro, 5 req/min login)             |
| Sesión timeout    | Middleware <code>session.timeout</code> (inactividad configurable)     |
| Roles             | Middleware <code>admin</code> valida rol antes de cualquier ruta admin |
| Contraseñas       | bcrypt hashing (Laravel Hash facade)                                   |
| Secreto 2FA       | Encriptado en DB con <code>encrypted</code> cast de Eloquent           |
| API Keys externas | Solo en <code>.env</code> (nunca en código fuente)                     |
| Dominios Sanctum  | <code>SANCTUM_STATEFUL_DOMAINS</code> lista blanca estricta            |

## API Reference — Hosting Platform Backend

Base URL: `https://<your-domain>/api`

Autenticación: **Cookie HttpOnly** `auth_token` (Laravel Sanctum). Enviar con `withCredentials: true` .  
CSRF: Header `X-XSRF-TOKEN` obtenido de la cookie `XSRF-TOKEN` .  
Formato: `Content-Type: application/json` .

---

## Índice

---

- Públicos / Sin autenticación
  - Autenticación
  - Portal Cliente
    - Dashboard
    - Perfil
    - Servicios
    - Game Servers
    - Gestión de archivos
    - Pagos y suscripciones
    - Facturas
    - Transacciones
    - Tickets de soporte
    - Dominios
    - Notificaciones
    - Fiscal / CFDI
    - Chat de soporte
    - Hosting (HestiaCP)
  - Panel Admin
    - Dashboard admin
    - Usuarios
    - Servicios admin
    - Facturas admin
    - Tickets admin
    - Agentes de soporte
    - Game Servers admin
    - Versiones de software
    - Pterodactyl (Eggs)
    - Cotizaciones
    - CFDI admin
    - Fiscal admin
    - Notificaciones admin
    - Chat admin
    - Catálogo admin
    - Blog admin
    - Documentación admin
    - Sistema y búsqueda
-

## Públicos / Sin autenticación

### Catálogo

| Método | Endpoint                                            | Descripción                         |
|--------|-----------------------------------------------------|-------------------------------------|
| GET    | <code>/categories</code>                            | Lista de categorías de servicios    |
| GET    | <code>/categories/with-plans</code>                 | Categorías con sus planes incluidos |
| GET    | <code>/categories/slug/{slug}</code>                | Categoría por slug                  |
| GET    | <code>/billing-cycles</code>                        | Ciclos de facturación disponibles   |
| GET    | <code>/billing-cycles/{uuid}</code>                 | Detalle de un ciclo                 |
| GET    | <code>/service-plans</code>                         | Todos los planes de servicio        |
| GET    | <code>/service-plans/{uuid}</code>                  | Detalle de un plan                  |
| GET    | <code>/service-plans/category/{categorySlug}</code> | Planes por categoría                |
| GET    | <code>/service-plans/add-ons/{planSlug}</code>      | Add-ons disponibles para un plan    |
| GET    | <code>/products</code>                              | Lista de productos                  |
| GET    | <code>/products/{uuid}</code>                       | Detalle de producto                 |
| GET    | <code>/products/service-type/{serviceType}</code>   | Productos por tipo de servicio      |
| GET    | <code>/marketing-services</code>                    | Servicios de marketing              |

### Game Eggs (catálogo de juegos)

| Método | Endpoint                              | Descripción                                                       |
|--------|---------------------------------------|-------------------------------------------------------------------|
| GET    | <code>/game-eggs?id={planUuid}</code> | Huevos (eggs) de Pterodactyl agrupados por nido para un plan dado |
| GET    | <code>/game-eggs/{id}</code>          | Detalle de un egg específico                                      |

### Software / Versiones

| Método | Endpoint                                     | Descripción                                                                           |
|--------|----------------------------------------------|---------------------------------------------------------------------------------------|
| GET    | <code>/software/{identifier}/versions</code> | Versiones disponibles de un software (ej. <code>vanilla</code> , <code>paper</code> ) |



## Blog

| Método | Endpoint                                           | Descripción               |
|--------|----------------------------------------------------|---------------------------|
| GET    | <code>/blog/posts</code>                           | Lista de posts publicados |
| GET    | <code>/blog/posts/featured</code>                  | Posts destacados          |
| GET    | <code>/blog/posts/{slug}</code>                    | Post por slug             |
| GET    | <code>/blog/categories</code>                      | Categorías del blog       |
| GET    | <code>/blog/categories/{categorySlug}/posts</code> | Posts de una categoría    |
| POST   | <code>/blog/subscribe</code>                       | Suscribirse al blog       |
| POST   | <code>/blog/unsubscribe/{uuid}</code>              | Cancelar suscripción      |

## Documentación pública

| Método | Endpoint                               | Descripción                            |
|--------|----------------------------------------|----------------------------------------|
| GET    | <code>/documentation</code>            | Lista de artículos de documentación    |
| GET    | <code>/documentation/{slug}</code>     | Artículo por slug                      |
| GET    | <code>/api-documentation</code>        | Índice de documentación API            |
| GET    | <code>/api-documentation/{slug}</code> | Artículo de doc API                    |
| POST   | <code>/documentation-requests</code>   | Solicitar un artículo de documentación |
| GET    | <code>/system-status</code>            | Estado del sistema                     |

## Cotizaciones públicas

| Método | Endpoint                                       | Descripción                         |
|--------|------------------------------------------------|-------------------------------------|
| GET    | <code>/quotations/public/{token}</code>        | Ver cotización por token (sin auth) |
| POST   | <code>/quotations/public/{token}/viewed</code> | Marcar cotización como vista        |

## Utilidades

| Método | Endpoint                          | Descripción                             |
|--------|-----------------------------------|-----------------------------------------|
| GET    | <code>/postal-codes/{code}</code> | Información de código postal SAT México |
| POST   | <code>/stripe/webhook</code>      | Webhook de eventos Stripe (sin auth)    |
| GET    | <code>/docs</code>                | JSON OpenAPI / Swagger                  |

## Autenticación

Throttle: 10 req/min en register/Google; 5 req/min en login/2FA.

## Endpoints públicos de auth

| Método | Endpoint                                     | Descripción                                                                          |
|--------|----------------------------------------------|--------------------------------------------------------------------------------------|
| POST   | <code>/auth/login</code>                     | Login con email y contraseña                                                         |
| POST   | <code>/auth/register</code>                  | Registro de nuevo usuario                                                            |
| POST   | <code>/auth/2fa/verify</code>                | Verificar código TOTP al iniciar sesión                                              |
| POST   | <code>/auth/google/callback</code>           | OAuth Google (portal cliente)                                                        |
| POST   | <code>/admin/auth/google/callback</code>     | OAuth Google (panel admin; valida rol)                                               |
| GET    | <code>/auth/username/check?username=X</code> | Verificar disponibilidad de username                                                 |
| POST   | <code>/auth/complete-profile</code>          | Completar perfil Google (requiere <code>setup_token</code> + <code>username</code> ) |
| POST   | <code>/forgot-password</code>                | Solicitar email de restablecimiento de contraseña                                    |
| POST   | <code>/reset-password</code>                 | Restablecer contraseña con token                                                     |

## Endpoints protegidos de auth

Requieren cookie de sesión válida ( `auth:sanctum` ).

| Método | Endpoint                          | Descripción                                        |
|--------|-----------------------------------|----------------------------------------------------|
| POST   | <code>/auth/logout</code>         | Cerrar sesión (invalida cookie)                    |
| GET    | <code>/auth/me</code>             | Usuario autenticado actual                         |
| GET    | <code>/user</code>                | Alias de <code>/auth/me</code>                     |
| POST   | <code>/auth/setup-username</code> | Asignar username a usuario autenticado sin uno     |
| GET    | <code>/2fa/status</code>          | Estado actual de 2FA                               |
| POST   | <code>/2fa/generate</code>        | Generar secreto TOTP (devuelve QR)                 |
| POST   | <code>/2fa/enable</code>          | Activar 2FA con código de verificación             |
| POST   | <code>/2fa/disable</code>         | Desactivar 2FA                                     |
| POST   | <code>/2fa/verify</code>          | Verificar código TOTP (cuando ya está autenticado) |

## Portal Cliente

Middleware: `auth:sanctum` + `session.timeout` en todas las rutas.

## Dashboard cliente

| Método | Endpoint                         | Descripción                                                                |
|--------|----------------------------------|----------------------------------------------------------------------------|
| GET    | <code>/dashboard/stats</code>    | Métricas del cliente (servicios activos, gasto mensual, tickets, dominios) |
| GET    | <code>/dashboard/services</code> | Servicios del cliente para el dashboard                                    |
| GET    | <code>/dashboard/activity</code> | Actividad reciente del cliente                                             |

## Perfil

| Método | Endpoint                                              | Descripción                                  |
|--------|-------------------------------------------------------|----------------------------------------------|
| GET    | <code>/profile</code>                                 | Obtener perfil del usuario                   |
| PUT    | <code>/profile</code>                                 | Actualizar datos del perfil                  |
| POST   | <code>/profile/avatar</code>                          | Subir foto de perfil                         |
| PUT    | <code>/profile/email</code>                           | Cambiar email                                |
| PUT    | <code>/profile/password</code>                        | Cambiar contraseña                           |
| POST   | <code>/profile/email/verification-notification</code> | Reenviar correo de verificación (throttled)  |
| GET    | <code>/profile/devices</code>                         | Sesiones activas del usuario                 |
| GET    | <code>/profile/security</code>                        | Resumen de seguridad (2FA, sesiones)         |
| DELETE | <code>/profile/account</code>                         | Eliminar cuenta                              |
| DELETE | <code>/profile/sessions/{uuid}</code>                 | Revocar sesión específica                    |
| POST   | <code>/profile/devices/revoke-others</code>           | Revocar todas las sesiones excepto la actual |

## Servicios

| Método | Endpoint                                                 | Descripción                                 |
|--------|----------------------------------------------------------|---------------------------------------------|
| GET    | <code>/services/plans</code>                             | Planes disponibles para contratación        |
| POST   | <code>/services/contract</code>                          | Contratar un nuevo servicio                 |
| GET    | <code>/services/user</code>                              | Servicios del usuario autenticado           |
| GET    | <code>/services/metrics</code>                           | Métricas de todos los servicios del usuario |
| GET    | <code>/services/{uuid}</code>                            | Detalle de un servicio                      |
| GET    | <code>/services/{uuid}/invoices</code>                   | Facturas de un servicio                     |
| PATCH  | <code>/services/{uuid}/configuration</code>              | Actualizar configuración del servicio       |
| PUT    | <code>/services/{uuid}/config</code>                     | Alias de configuración                      |
| POST   | <code>/services/{uuid}/cancel</code>                     | Solicitar cancelación                       |
| POST   | <code>/services/{uuid}/suspend</code>                    | Suspender servicio                          |
| POST   | <code>/services/{uuid}/reactivate</code>                 | Reactivar servicio                          |
| GET    | <code>/services/{uuid}/backups</code>                    | Listar backups                              |
| POST   | <code>/services/{uuid}/backups</code>                    | Crear backup                                |
| POST   | <code>/services/{uuid}/backups/{backupId}/restore</code> | Restaurar backup                            |

## Game Servers cliente

| Método | Endpoint                                                    | Descripción                                                                                              |
|--------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| GET    | <code>/services/game-servers/{nest_id}/eggs</code>          | Eggs disponibles por nido                                                                                |
| GET    | <code>/services/{uuid}/game-server/startup</code>           | Configuración de arranque del servidor                                                                   |
| GET    | <code>/services/{uuid}/game-server/usage</code>             | Métricas en tiempo real (CPU, RAM, red, disco)                                                           |
| POST   | <code>/services/{uuid}/game-server/power</code>             | Control de energía ( <code>start</code> / <code>stop</code> / <code>restart</code> / <code>kill</code> ) |
| GET    | <code>/services/{uuid}/game-server/websocket</code>         | Credenciales WebSocket de Wings para la consola                                                          |
| POST   | <code>/services/{uuid}/game-server/command</code>           | Enviar comando a la consola                                                                              |
| GET    | <code>/services/{uuid}/game-server/software-options</code>  | Opciones de software disponibles (eggs del plan)                                                         |
| GET    | <code>/services/{uuid}/game-server/configuration</code>     | Configuración actual (software, server.properties, EULA, Java)                                           |
| PATCH  | <code>/services/{uuid}/game-server/software</code>          | Cambiar software / versión                                                                               |
| PATCH  | <code>/services/{uuid}/game-server/server-properties</code> | Actualizar <code>server.properties</code>                                                                |
| POST   | <code>/services/{uuid}/game-server/restart-required</code>  | Marcar que se requiere reinicio                                                                          |
| GET    | <code>/services/{uuid}/game-server/eula</code>              | Estado de aceptación de EULA (Minecraft Java)                                                            |
| POST   | <code>/services/{uuid}/game-server/eula/accept</code>       | Aceptar EULA de Minecraft                                                                                |
| POST   | <code>/services/{uuid}/game-server/fix-java</code>          | Corregir versión de Java del servidor                                                                    |
| GET    | <code>/services/{uuid}/game-server/java-check</code>        | Verificar compatibilidad de Java                                                                         |
| POST   | <code>/services/{uuid}/game-server/java-autofix</code>      | Auto-corrregir compatibilidad de Java                                                                    |
| GET    | <code>/services/{uuid}/game-server/java-requirements</code> | Requisitos de Java para el egg activo                                                                    |

## Gestión de archivos

| Método | Endpoint                                     | Descripción                   |
|--------|----------------------------------------------|-------------------------------|
| GET    | <code>/services/{uuid}/files/list</code>     | Listar archivos y directorios |
| GET    | <code>/services/{uuid}/files/upload</code>   | Obtener URL de subida (Wings) |
| POST   | <code>/services/{uuid}/files/delete</code>   | Eliminar archivos             |
| GET    | <code>/services/{uuid}/files/download</code> | Obtener URL de descarga       |

## Pagos y suscripciones

| Método | Endpoint                                | Descripción                          |
|--------|-----------------------------------------|--------------------------------------|
| GET    | <code>/payments/methods</code>          | Métodos de pago guardados            |
| POST   | <code>/payments/methods</code>          | Agregar método de pago               |
| PUT    | <code>/payments/methods/{id}</code>     | Actualizar (ej. predeterminado)      |
| DELETE | <code>/payments/methods/{id}</code>     | Eliminar método de pago              |
| POST   | <code>/payments/setup-intent</code>     | Crear SetupIntent de Stripe          |
| POST   | <code>/payments/intent</code>           | Alias de setup-intent                |
| POST   | <code>/payments/process</code>          | Procesar pago de una factura         |
| GET    | <code>/payments/stats</code>            | Estadísticas de pagos                |
| GET    | <code>/payments/transactions</code>     | Historial de transacciones via pagos |
| GET    | <code>/subscriptions</code>             | Suscripciones activas del usuario    |
| POST   | <code>/subscriptions</code>             | Crear suscripción                    |
| GET    | <code>/subscriptions/{id}</code>        | Detalle de suscripción               |
| POST   | <code>/subscriptions/{id}/cancel</code> | Cancelar suscripción                 |
| POST   | <code>/subscriptions/{id}/resume</code> | Reanudar suscripción                 |

## Facturas

| Método | Endpoint                                  | Descripción                                             |
|--------|-------------------------------------------|---------------------------------------------------------|
| GET    | <code>/invoices</code>                    | Lista de facturas (soporta paginación y filtros)        |
| GET    | <code>/invoices/stats</code>              | Estadísticas de facturación                             |
| GET    | <code>/invoices/{uuid}</code>             | Detalle de factura                                      |
| GET    | <code>/invoices/{uuid}/receipt</code>     | Descargar comprobante de pago (blob PDF)                |
| GET    | <code>/invoices/{uuid}/pdf</code>         | Descargar CFDI en PDF                                   |
| GET    | <code>/invoices/{uuid}/xml</code>         | Descargar CFDI en XML                                   |
| PUT    | <code>/invoices/{uuid}/fiscal-data</code> | Actualizar RFC/datos CFDI (ventana de 72h tras el pago) |

## Transacciones

| Método | Endpoint                          | Descripción                                       |
|--------|-----------------------------------|---------------------------------------------------|
| GET    | <code>/transactions</code>        | Historial de transacciones                        |
| GET    | <code>/transactions/stats</code>  | Estadísticas de transacciones                     |
| GET    | <code>/transactions/recent</code> | Transacciones recientes ( <code>?limit=N</code> ) |
| GET    | <code>/transactions/{uuid}</code> | Detalle de transacción                            |

## Tickets de soporte

| Método | Endpoint                           | Descripción                         |
|--------|------------------------------------|-------------------------------------|
| GET    | <code>/tickets</code>              | Lista de tickets del usuario        |
| POST   | <code>/tickets</code>              | Crear nuevo ticket                  |
| GET    | <code>/tickets/stats</code>        | Estadísticas de tickets             |
| GET    | <code>/tickets/{uuid}</code>       | Detalle de ticket                   |
| POST   | <code>/tickets/{uuid}/reply</code> | Agregar respuesta (admite adjuntos) |
| PUT    | <code>/tickets/{uuid}/close</code> | Cerrar ticket                       |

## Dominios

| Método | Endpoint                                 | Descripción                            |
|--------|------------------------------------------|----------------------------------------|
| GET    | <code>/domains</code>                    | Dominios del usuario                   |
| POST   | <code>/domains</code>                    | Registrar dominio                      |
| GET    | <code>/domains/stats</code>              | Estadísticas de dominios               |
| POST   | <code>/domains/check-availability</code> | Verificar disponibilidad de un dominio |
| GET    | <code>/domains/{uuid}</code>             | Detalle de dominio                     |
| PUT    | <code>/domains/{uuid}</code>             | Actualizar dominio                     |
| POST   | <code>/domains/{uuid}/renew</code>       | Renovar dominio                        |

## Notificaciones cliente

| Método | Endpoint                                        | Descripción                  |
|--------|-------------------------------------------------|------------------------------|
| GET    | <code>/notifications</code>                     | Lista de notificaciones      |
| GET    | <code>/notifications/unread-count</code>        | Conteo de no leídas          |
| GET    | <code>/notifications/preferences</code>         | Preferencias de notificación |
| PUT    | <code>/notifications/preferences</code>         | Actualizar preferencias      |
| PUT    | <code>/notifications/{notification}/read</code> | Marcar como leída            |
| PUT    | <code>/notifications/mark-all-read</code>       | Marcar todas como leídas     |
| DELETE | <code>/notifications/{notification}</code>      | Eliminar notificación        |

## Fiscal / CFDI cliente

| Método | Endpoint                                         | Descripción                        |
|--------|--------------------------------------------------|------------------------------------|
| GET    | <code>/fiscal/regimes</code>                     | Regímenes fiscales SAT disponibles |
| GET    | <code>/fiscal/cfdi-uses</code>                   | Usos de CFDI SAT disponibles       |
| GET    | <code>/fiscal/profiles</code>                    | Perfiles fiscales del usuario      |
| POST   | <code>/fiscal/profiles</code>                    | Crear perfil fiscal                |
| GET    | <code>/fiscal/profiles/{uuid}</code>             | Detalle de perfil                  |
| PUT    | <code>/fiscal/profiles/{uuid}</code>             | Actualizar perfil                  |
| DELETE | <code>/fiscal/profiles/{uuid}</code>             | Eliminar perfil                    |
| PUT    | <code>/fiscal/profiles/{uuid}/set-default</code> | Establecer perfil predeterminado   |

## Chat de soporte

| Método | Endpoint                             | Descripción                                 |
|--------|--------------------------------------|---------------------------------------------|
| GET    | <code>/chat/support-room</code>      | Obtener o crear sala de soporte del usuario |
| GET    | <code>/chat/unread-count</code>      | Mensajes no leídos                          |
| GET    | <code>/chat/history</code>           | Historial de chats                          |
| GET    | <code>/chat/{ticket}/messages</code> | Mensajes de un chat                         |
| POST   | <code>/chat/{ticket}/messages</code> | Enviar mensaje                              |
| PUT    | <code>/chat/{ticket}/read</code>     | Marcar mensajes como leídos                 |
| PUT    | <code>/chat/{ticket}/close</code>    | Cerrar sala de chat                         |

## Búsqueda cliente

| Método | Endpoint                       | Descripción                    |
|--------|--------------------------------|--------------------------------|
| GET    | <code>/search?q={query}</code> | Búsqueda en el portal cliente  |
| GET    | <code>/search/popular</code>   | Términos de búsqueda populares |

## Hosting (HestiaCP)

Requieren `auth:sanctum` + `session.timeout` .

| Método | Endpoint                                      | Descripción                     |
|--------|-----------------------------------------------|---------------------------------|
| GET    | <code>/hosting/{uuid}/info</code>             | Información general del hosting |
| GET    | <code>/hosting/{uuid}/files</code>            | Archivos del hosting            |
| GET    | <code>/hosting/{uuid}/databases</code>        | Bases de datos                  |
| POST   | <code>/hosting/{uuid}/databases</code>        | Crear base de datos             |
| DELETE | <code>/hosting/{uuid}/databases/{db}</code>   | Eliminar base de datos          |
| GET    | <code>/hosting/{uuid}/emails</code>           | Cuentas de email                |
| POST   | <code>/hosting/{uuid}/emails</code>           | Crear cuenta de email           |
| DELETE | <code>/hosting/{uuid}/emails/{account}</code> | Eliminar cuenta de email        |
| GET    | <code>/hosting/{uuid}/domains</code>          | Dominios del hosting            |
| POST   | <code>/hosting/{uuid}/domains</code>          | Agregar dominio                 |
| DELETE | <code>/hosting/{uuid}/domains/{domain}</code> | Eliminar dominio                |
| GET    | <code>/hosting/{uuid}/stats</code>            | Estadísticas de uso             |

## Panel Admin

Middleware: `auth:sanctum` + `session.timeout` + `admin` .  
 Todas las rutas usan el prefijo `/admin` .

### Dashboard admin

| Método | Endpoint                            | Descripción                   |
|--------|-------------------------------------|-------------------------------|
| GET    | <code>/admin/dashboard/stats</code> | Métricas globales del sistema |

### Usuarios admin

| Método | Endpoint                              | Descripción                                                                                                                             |
|--------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>/admin/users</code>             | Lista de usuarios (filtros: <code>search</code> , <code>status</code> , <code>role</code> , <code>page</code> , <code>per_page</code> ) |
| POST   | <code>/admin/users</code>             | Crear usuario                                                                                                                           |
| PUT    | <code>/admin/users/{id}</code>        | Actualizar usuario                                                                                                                      |
| DELETE | <code>/admin/users/{id}</code>        | Eliminar usuario                                                                                                                        |
| PUT    | <code>/admin/users/{id}/status</code> | Cambiar estado del usuario                                                                                                              |



## Servicios admin

| Método | Endpoint                                             | Descripción                                                                                                                                 |
|--------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>/admin/services</code>                         | Lista de servicios (filtros: <code>search</code> , <code>status</code> , <code>plan_id</code> , <code>page</code> , <code>per_page</code> ) |
| POST   | <code>/admin/services</code>                         | Crear servicio                                                                                                                              |
| GET    | <code>/admin/services/{uuid}</code>                  | Detalle de servicio                                                                                                                         |
| PUT    | <code>/admin/services/{uuid}</code>                  | Actualizar servicio                                                                                                                         |
| DELETE | <code>/admin/services/{uuid}</code>                  | Eliminar servicio                                                                                                                           |
| PUT    | <code>/admin/services/{uuid}/status</code>           | Cambiar estado ( <code>active</code> / <code>suspended</code> / <code>cancelled</code> )                                                    |
| GET    | <code>/admin/services/{uuid}/support-overview</code> | Vista de soporte del servicio (tickets relacionados)                                                                                        |

## Facturas admin

| Método | Endpoint                                        | Descripción                   |
|--------|-------------------------------------------------|-------------------------------|
| GET    | <code>/admin/invoices</code>                    | Lista de facturas             |
| GET    | <code>/admin/invoices/stats</code>              | Estadísticas de facturación   |
| POST   | <code>/admin/invoices</code>                    | Crear factura                 |
| PUT    | <code>/admin/invoices/{id}</code>               | Actualizar factura            |
| DELETE | <code>/admin/invoices/{id}</code>               | Eliminar factura              |
| PUT    | <code>/admin/invoices/{id}/status</code>        | Cambiar estado                |
| POST   | <code>/admin/invoices/{id}/mark-paid</code>     | Marcar como pagada            |
| POST   | <code>/admin/invoices/{id}/send-reminder</code> | Enviar recordatorio de pago   |
| POST   | <code>/admin/invoices/{id}/cancel</code>        | Cancelar factura              |
| GET    | <code>/admin/invoices/{uuid}/receipt</code>     | Descargar comprobante (admin) |
| GET    | <code>/admin/invoices/{serviceId}</code>        | Facturas por servicio         |

## Tickets admin

| Método | Endpoint                                  | Descripción           |
|--------|-------------------------------------------|-----------------------|
| GET    | <code>/admin/tickets</code>               | Lista de tickets      |
| POST   | <code>/admin/tickets</code>               | Crear ticket          |
| GET    | <code>/admin/tickets/stats</code>         | Estadísticas          |
| GET    | <code>/admin/tickets/categories</code>    | Categorías de tickets |
| GET    | <code>/admin/support-agents</code>        | Agentes disponibles   |
| GET    | <code>/admin/tickets/{id}</code>          | Detalle de ticket     |
| PUT    | <code>/admin/tickets/{id}</code>          | Actualizar ticket     |
| DELETE | <code>/admin/tickets/{id}</code>          | Eliminar ticket       |
| PUT    | <code>/admin/tickets/{id}/status</code>   | Cambiar estado        |
| PUT    | <code>/admin/tickets/{id}/priority</code> | Cambiar prioridad     |
| POST   | <code>/admin/tickets/{id}/assign</code>   | Asignar a agente      |
| POST   | <code>/admin/tickets/{id}/reply</code>    | Agregar respuesta     |

## Agentes de soporte

| Método | Endpoint                                                | Descripción                        |
|--------|---------------------------------------------------------|------------------------------------|
| GET    | <code>/admin/tickets/agents</code>                      | Lista de agentes                   |
| POST   | <code>/admin/tickets/agents</code>                      | Crear agente                       |
| GET    | <code>/admin/tickets/agents/statistics</code>           | Estadísticas de agentes            |
| GET    | <code>/admin/tickets/agents/recommended</code>          | Agente recomendado para asignación |
| GET    | <code>/admin/tickets/agents/{uuid}</code>               | Detalle de agente                  |
| PUT    | <code>/admin/tickets/agents/{uuid}</code>               | Actualizar agente                  |
| DELETE | <code>/admin/tickets/agents/{uuid}</code>               | Eliminar agente                    |
| POST   | <code>/admin/tickets/agents/{uuid}/assign-ticket</code> | Asignar ticket a agente            |
| GET    | <code>/admin/tickets/agents/{uuid}/tickets</code>       | Tickets asignados a un agente      |

## Game Servers admin

| Método | Endpoint                                             | Descripción                           |
|--------|------------------------------------------------------|---------------------------------------|
| GET    | <code>/admin/game-servers</code>                     | Lista de game servers                 |
| GET    | <code>/admin/game-servers/{uuid}</code>              | Detalle de game server                |
| POST   | <code>/admin/game-servers/{id}/provision</code>      | Provisionar servidor en Pterodactyl   |
| POST   | <code>/admin/game-servers/{id}/suspend</code>        | Suspender servidor                    |
| POST   | <code>/admin/game-servers/{id}/unsuspend</code>      | Desuspender servidor                  |
| POST   | <code>/admin/game-servers/{id}/reinstall</code>      | Reinstalar servidor                   |
| DELETE | <code>/admin/game-servers/{id}</code>                | Terminar y eliminar servidor          |
| GET    | <code>/admin/game-servers/{id}/websocket</code>      | Credenciales WebSocket (bypass admin) |
| GET    | <code>/admin/game-servers/{id}/usage</code>          | Métricas en tiempo real               |
| POST   | <code>/admin/game-servers/{id}/power</code>          | Control de energía                    |
| POST   | <code>/admin/game-servers/{id}/command</code>        | Enviar comando                        |
| GET    | <code>/admin/game-servers/{id}/files/list</code>     | Listar archivos                       |
| GET    | <code>/admin/game-servers/{id}/files/upload</code>   | URL de subida                         |
| POST   | <code>/admin/game-servers/{id}/files/delete</code>   | Eliminar archivos                     |
| GET    | <code>/admin/game-servers/{id}/files/download</code> | URL de descarga                       |

## Versiones de software admin

| Método | Endpoint                                        | Descripción                                                                                     |
|--------|-------------------------------------------------|-------------------------------------------------------------------------------------------------|
| GET    | <code>/admin/game-versions</code>               | Lista de versiones (filtros: <code>software</code> , <code>enabled</code> , <code>page</code> ) |
| POST   | <code>/admin/game-versions</code>               | Crear versión                                                                                   |
| PUT    | <code>/admin/game-versions/{id}</code>          | Actualizar versión                                                                              |
| DELETE | <code>/admin/game-versions/{id}</code>          | Eliminar versión                                                                                |
| POST   | <code>/admin/game-versions/bulk/{action}</code> | Acciones en lote ( <code>enable</code> / <code>disable</code> / <code>delete</code> )           |

## Pterodactyl (Eggs) admin

| Método | Endpoint                                         | Descripción                              |
|--------|--------------------------------------------------|------------------------------------------|
| GET    | <code>/admin/pterodactyl/eggs</code>             | Lista de eggs registrados                |
| PATCH  | <code>/admin/pterodactyl/eggs/{id}</code>        | Actualizar egg                           |
| POST   | <code>/admin/pterodactyl/eggs/{id}/toggle</code> | Activar / desactivar egg                 |
| POST   | <code>/admin/pterodactyl/eggs/sync</code>        | Sincronizar eggs desde Pterodactyl Panel |

## Cotizaciones admin

| Método | Endpoint                                                   | Descripción                    |
|--------|------------------------------------------------------------|--------------------------------|
| GET    | <code>/admin/quotations</code>                             | Lista de cotizaciones          |
| POST   | <code>/admin/quotations</code>                             | Crear cotización               |
| GET    | <code>/admin/quotations/{quotation}</code>                 | Detalle                        |
| PUT    | <code>/admin/quotations/{quotation}</code>                 | Actualizar                     |
| DELETE | <code>/admin/quotations/{quotation}</code>                 | Eliminar                       |
| POST   | <code>/admin/quotations/{quotation}/send</code>            | Enviar cotización al cliente   |
| POST   | <code>/admin/quotations/{quotation}/accept</code>          | Marcar como aceptada           |
| POST   | <code>/admin/quotations/{quotation}/reject</code>          | Marcar como rechazada          |
| POST   | <code>/admin/quotations/{quotation}/reopen</code>          | Reabrir cotización             |
| POST   | <code>/admin/quotations/{quotation}/revision</code>        | Crear revisión / nueva versión |
| POST   | <code>/admin/quotations/{quotation}/regenerate-link</code> | Regenerar link público         |

## CFDI admin

| Método | Endpoint                                        | Descripción                                            |
|--------|-------------------------------------------------|--------------------------------------------------------|
| GET    | <code>/admin/cfdi</code>                        | Lista de CFDIs con filtros                             |
| GET    | <code>/admin/cfdi/stats</code>                  | Estadísticas de timbrado                               |
| GET    | <code>/admin/cfdi/{id}</code>                   | Detalle de CFDI                                        |
| POST   | <code>/admin/cfdi/{id}/retry</code>             | Reintentar timbrado fallido                            |
| POST   | <code>/admin/cfdi/{id}/cancel</code>            | Cancelar CFDI ante el SAT                              |
| GET    | <code>/admin/cfdi/{id}/download/{format}</code> | Descargar CFDI ( <code>pdf</code> o <code>xml</code> ) |

## Fiscal admin

| Método | Endpoint                                           | Descripción                   |
|--------|----------------------------------------------------|-------------------------------|
| GET    | <code>/admin/fiscal/regimes</code>                 | Catálogo de regímenes SAT     |
| PUT    | <code>/admin/fiscal/regimes/{code}/toggle</code>   | Activar / desactivar régimen  |
| GET    | <code>/admin/fiscal/cfdi-uses</code>               | Catálogo de usos CFDI SAT     |
| PUT    | <code>/admin/fiscal/cfdi-uses/{code}/toggle</code> | Activar / desactivar uso      |
| GET    | <code>/admin/fiscal/profiles</code>                | Perfiles fiscales de clientes |
| GET    | <code>/admin/fiscal/profiles/{uuid}</code>         | Detalle de perfil fiscal      |

## Notificaciones admin

| Método | Endpoint                                                   | Descripción                              |
|--------|------------------------------------------------------------|------------------------------------------|
| GET    | <code>/admin/notifications/dashboard</code>                | Dashboard de notificaciones              |
| GET    | <code>/admin/notifications</code>                          | Lista de notificaciones                  |
| GET    | <code>/admin/notifications/stats</code>                    | Estadísticas                             |
| POST   | <code>/admin/notifications/broadcast</code>                | Enviar notificación a todos los usuarios |
| POST   | <code>/admin/notifications/send-to-user/{user}</code>      | Enviar notificación a usuario específico |
| PUT    | <code>/admin/notifications/mark-all-read</code>            | Marcar todas como leídas                 |
| PUT    | <code>/admin/notifications/archive-all-read</code>         | Archivar leídas                          |
| DELETE | <code>/admin/notifications/archived</code>                 | Eliminar archivadas                      |
| PUT    | <code>/admin/notifications/{notification}/read</code>      | Marcar como leída                        |
| PUT    | <code>/admin/notifications/{notification}/archive</code>   | Archivar                                 |
| PUT    | <code>/admin/notifications/{notification}/unarchive</code> | Desarchivar                              |
| DELETE | <code>/admin/notifications/{notification}</code>           | Eliminar notificación                    |

## Chat admin

| Método | Endpoint                                     | Descripción           |
|--------|----------------------------------------------|-----------------------|
| GET    | <code>/admin/chat/active-rooms</code>        | Salas de chat activas |
| GET    | <code>/admin/chat/all-rooms</code>           | Todas las salas       |
| GET    | <code>/admin/chat/stats</code>               | Estadísticas de chat  |
| GET    | <code>/admin/chat/unread-count</code>        | Mensajes no leídos    |
| GET    | <code>/admin/chat/{chatRoom}/messages</code> | Mensajes de una sala  |
| POST   | <code>/admin/chat/{chatRoom}/messages</code> | Enviar mensaje        |
| PUT    | <code>/admin/chat/{chatRoom}/assign</code>   | Asignar sala a agente |
| PUT    | <code>/admin/chat/{chatRoom}/close</code>    | Cerrar sala           |
| PUT    | <code>/admin/chat/{chatRoom}/reopen</code>   | Reabrir sala          |

## Catálogo admin

| Método          | Endpoint                                            | Descripción                         |
|-----------------|-----------------------------------------------------|-------------------------------------|
| GET/POST        | <code>/admin/categories</code>                      | Lista / Crear categorías            |
| PUT/DELETE      | <code>/admin/categories/{uuid}</code>               | Actualizar / Eliminar               |
| GET/POST        | <code>/admin/billing-cycles</code>                  | Lista / Crear ciclos de facturación |
| PUT/DELETE      | <code>/admin/billing-cycles/{uuid}</code>           | Actualizar / Eliminar               |
| GET/POST        | <code>/admin/service-plans</code>                   | Lista / Crear planes                |
| GET             | <code>/admin/service-plans/{uuid}</code>            | Detalle                             |
| PUT/DELETE      | <code>/admin/service-plans/{uuid}</code>            | Actualizar / Eliminar               |
| POST            | <code>/admin/service-plans/bulk/{action}</code>     | Acciones en lote                    |
| GET/POST        | <code>/admin/add-ons</code>                         | Lista / Crear add-ons               |
| GET             | <code>/admin/add-ons/{uuid}</code>                  | Detalle                             |
| PUT/DELETE      | <code>/admin/add-ons/{uuid}</code>                  | Actualizar / Eliminar               |
| POST            | <code>/admin/add-ons/{uuid}/attach-to-plan</code>   | Asociar add-on a plan               |
| POST            | <code>/admin/add-ons/{uuid}/detach-from-plan</code> | Desasociar                          |
| POST/PUT/DELETE | <code>/admin/products/{uuid}</code>                 | CRUD de productos                   |

## Blog admin

| Método              | Endpoint                                      | Descripción                   |
|---------------------|-----------------------------------------------|-------------------------------|
| GET/POST/PUT/DELETE | <code>/admin/blog-categories</code>           | CRUD de categorías del blog   |
| GET/POST/PUT/DELETE | <code>/admin/blog-posts</code>                | CRUD de posts (apiResource)   |
| POST                | <code>/admin/blog/upload-image</code>         | Subir imagen al blog          |
| GET/DELETE          | <code>/admin/blog-subscriptions</code>        | Lista / Eliminar suscriptores |
| GET                 | <code>/admin/blog-subscriptions/{uuid}</code> | Detalle de suscriptor         |

## Documentación admin

| Método         | Endpoint                                                      | Descripción                         |
|----------------|---------------------------------------------------------------|-------------------------------------|
| GET/POST       | <code>/admin/documentation</code>                             | Lista / Crear artículos             |
| GET/PUT/DELETE | <code>/admin/documentation/{uuid}</code>                      | Detalle / Actualizar / Eliminar     |
| GET/POST       | <code>/admin/api-documentation</code>                         | Lista / Crear artículos de API docs |
| GET/PUT/DELETE | <code>/admin/api-documentation/{uuid}</code>                  | Detalle / Actualizar / Eliminar     |
| GET            | <code>/admin/documentation-requests</code>                    | Lista de solicitudes de docs        |
| GET            | <code>/admin/documentation-requests/{id}</code>               | Detalle                             |
| PUT            | <code>/admin/documentation-requests/{id}/mark-resolved</code> | Marcar como resuelta                |
| DELETE         | <code>/admin/documentation-requests/{id}</code>               | Eliminar                            |

## Sistema y búsqueda admin

| Método         | Endpoint                                 | Descripción                                           |
|----------------|------------------------------------------|-------------------------------------------------------|
| GET            | <code>/admin/search?q={query}</code>     | Búsqueda global (throttle: rate <code>search</code> ) |
| GET            | <code>/admin/search/popular</code>       | Términos populares de búsqueda                        |
| GET/POST       | <code>/admin/system-status</code>        | Lista / Crear servicios de estado                     |
| GET/PUT/DELETE | <code>/admin/system-status/{uuid}</code> | Detalle / Actualizar / Eliminar                       |

## Códigos de respuesta comunes

| Código           | Significado                                          |
|------------------|------------------------------------------------------|
| <code>200</code> | OK — respuesta exitosa                               |
| <code>201</code> | Created — recurso creado                             |
| <code>204</code> | No Content — operación exitosa sin cuerpo            |
| <code>400</code> | Bad Request — validación fallida                     |
| <code>401</code> | Unauthorized — sesión inexistente o expirada         |
| <code>403</code> | Forbidden — sin permisos suficientes                 |
| <code>404</code> | Not Found — recurso no encontrado                    |
| <code>419</code> | CSRF Token Mismatch — cookie XSRF inválida           |
| <code>422</code> | Unprocessable Entity — errores de validación Laravel |
| <code>429</code> | Too Many Requests — throttle superado                |
| <code>500</code> | Server Error — error interno                         |

## Formato de error estándar

```
{
  "message": "The given data was invalid.",
  "errors": {
    "email": ["The email field is required."],
    "password": ["The password must be at least 8 characters."]
  }
}
```

## PARTE II

# Frontend

React 19 · TypeScript · Vite · Tailwind CSS · React Query

## ROKE Industries — Hosting Platform Frontend

---

### Descripción general

---

Aplicación React/TypeScript que implementa el portal de clientes y el panel de administración de ROKE Industries. Incluye gestión de servicios de hosting, game servers, facturación electrónica (CFDI 4.0 SAT México), soporte, cotizaciones y blog.

---



## Stack tecnológico

| Componente           | Tecnología                                         |
|----------------------|----------------------------------------------------|
| Framework            | React 19                                           |
| Lenguaje             | TypeScript 6                                       |
| Bundler              | Vite 6                                             |
| Gestor de paquetes   | pnpm 10                                            |
| Estilos              | Tailwind CSS v4                                    |
| Estado del servidor  | TanStack React Query v5                            |
| Formularios          | react-hook-form + Zod                              |
| Routing              | react-router-dom v7                                |
| UI base              | Radix UI + shadcn/ui                               |
| Animaciones          | Framer Motion                                      |
| Editor rich text     | Tiptap v3                                          |
| Gráficas             | Recharts                                           |
| Pagos                | Stripe ( <a href="#">@stripe/react-stripe-js</a> ) |
| Auth Google          | <a href="#">@react-oauth/google</a>                |
| WebSockets           | Laravel Echo + Pusher JS → Laravel Reverb          |
| Internacionalización | i18next + react-i18next                            |
| Monitoreo            | Sentry ( <a href="#">@sentry/react</a> )           |
| Testing              | Vitest + Testing Library + MSW + Playwright        |
| Storybook            | Componentes UI aislados                            |

## Builds

La aplicación genera **dos builds independientes**:

| Build                   | Entry point                      | Directorio de salida         | Uso                       |
|-------------------------|----------------------------------|------------------------------|---------------------------|
| Portal de clientes      | <a href="#">index.html</a>       | <a href="#">dist-portal/</a> | Clientes finales          |
| Panel de administración | <a href="#">index-admin.html</a> | <a href="#">dist-admin/</a>  | Administradores y soporte |

## Alias de paths (Vite)

| Alias           | Directorio real     |
|-----------------|---------------------|
| @               | src/                |
| @core           | src/core/           |
| @application    | src/application/    |
| @infrastructure | src/infrastructure/ |
| @presentation   | src/presentation/   |
| @shared         | src/shared/         |

## Variables de entorno ( `.env` )

| Variable               | Descripción                                                    |
|------------------------|----------------------------------------------------------------|
| VITE_API_BASE_URL      | URL base de la API (ej. <code>https://api.roke.mx/api</code> ) |
| VITE_API_URL           | URL raíz para broadcasting auth (sin <code>/api</code> )       |
| VITE_REVERB_HOST       | Host del servidor Reverb WebSocket                             |
| VITE_REVERB_PORT       | Puerto Reverb                                                  |
| VITE_REVERB_SCHEME     | WS o WSS                                                       |
| VITE_STRIPE_PUBLIC_KEY | Clave pública Stripe                                           |
| VITE_GOOGLE_CLIENT_ID  | Client ID Google OAuth                                         |
| VITE_SENTRY_DSN        | DSN de Sentry                                                  |

## Inicio rápido

```
pnpm install
pnpm dev           # Desarrollo (portal cliente)
pnpm dev:admin     # Desarrollo (panel admin)
pnpm build         # Build producción (ambos)
pnpm test          # Tests unitarios con Vitest
pnpm test:e2e      # Tests E2E con Playwright
pnpm storybook     # Servidor de Storybook
```

## Documentación detallada

- [Arquitectura técnica](#) — estructura de capas, routing, estado, WebSockets, React Query
- [Referencia de Hooks](#) — todos los hooks de datos con parámetros y retornos
- [Guía de funcionalidades](#) — módulos del sistema: checkout, game servers, facturación, soporte, admin



# Arquitectura técnica — Hosting Platform Frontend

## Estructura de capas (Clean Architecture)

```

src/
├── core/                                # Entidades e interfaces de dominio
│   └── (tipos base, modelos de API)
├── application/
│   ├── context/
│   │   ├── AuthContext.tsx            # Contexto de autenticación global
│   │   └── ThemeContext.tsx          # Tema (dark/light)
│   └── hooks/                          # Toda la lógica de datos (React Query)
│       ├── useAuth.ts
│       ├── useGameServer.ts
│       ├── useInvoices.ts
│       ├── useCheckout.ts
│       └── ...
├── infrastructure/
│   ├── api/
│   │   └── apiClient.ts                # Instancia Axios base (Sanctum + CSRF)
│   └── services/                       # Llamadas HTTP al backend
│       ├── authService.ts
│       ├── serviceService.ts
│       ├── adminServicesService.ts
│       ├── invoiceService.ts
│       ├── echoService.ts             # WebSockets (Laravel Echo)
│       └── ...
├── presentation/
│   ├── components/
│   │   ├── features/                  # Componentes de dominio
│   │   │   ├── checkout/
│   │   │   ├── services/
│   │   │   │   └── game-server/ # GameServerDetail, ModsManager, etc.
│   │   │   ├── tickets/
│   │   │   ├── invoices/
│   │   │   └── admin/
│   │   └── ui/                        # Primitivas (shadcn/ui + Radix)
│   ├── layouts/
│   │   ├── AppPortal.tsx              # Router del portal cliente
│   │   └── AppAdmin.tsx               # Router del panel admin
│   └── pages/
│       ├── auth/
│       ├── client/
│       └── admin/
└── shared/
    └── constants/

```

```
|   └─ queryConfig.ts      # Políticas de caché React Query
|   └─ utils/              # Formatters, validaciones, geo, i18n, CFDI
```

## Routing

### Portal de Clientes — `AppPortal.tsx`

#### Rutas públicas (sin sesión)

| Ruta                                | Componente                       | Descripción                               |
|-------------------------------------|----------------------------------|-------------------------------------------|
| <code>/login</code>                 | <code>LoginPage</code>           | Login con email/contraseña y Google OAuth |
| <code>/register</code>              | <code>RegisterPage</code>        | Registro de nuevo usuario                 |
| <code>/verify-2fa</code>            | <code>Verify2FAPage</code>       | Verificación de código TOTP               |
| <code>/forgot-password</code>       | <code>ForgotPasswordPage</code>  | Solicitud de reset de contraseña          |
| <code>/reset-password</code>        | <code>ResetPasswordPage</code>   | Formulario con token de reset             |
| <code>/auth/complete-profile</code> | <code>CompleteProfilePage</code> | Completa perfil de Google (setup_token)   |
| <code>/auth/setup-username</code>   | <code>SetupUsernamePage</code>   | Asignar username a usuario sin uno        |

#### Rutas protegidas ( `/client/*` )

| Ruta                                       | Componente                         | Descripción                                                      |
|--------------------------------------------|------------------------------------|------------------------------------------------------------------|
| <code>/client/</code>                      | <code>ClientDashboardPage</code>   | Dashboard principal del cliente                                  |
| <code>/client/services</code>              | <code>ClientServicesPage</code>    | Lista de servicios                                               |
| <code>/client/services/:uuid</code>        | <code>ServiceDetailPage</code>     | Detalle de servicio (redirige a game server o hosting si aplica) |
| <code>/client/hosting/:uuid</code>         | <code>HostingDetailPage</code>     | Detalle de hosting HestiaCP                                      |
| <code>/client/services/:uuid/manage</code> | <code>ServiceManagementPage</code> | Configuración avanzada del servicio                              |
| <code>/client/invoices</code>              | <code>ClientInvoicesPage</code>    | Facturación (comprobantes, métodos de pago, transacciones)       |
| <code>/client/tickets</code>               | <code>ClientTicketsPage</code>     | Tickets de soporte                                               |
| <code>/client/profile</code>               | <code>ClientProfilePage</code>     | Perfil, seguridad, 2FA, sesiones                                 |
| <code>/client/catalog</code>               | <code>ContractServicePage</code>   | Catálogo de planes para contratar                                |
| <code>/client/checkout</code>              | <code>CheckoutPage</code>          | Flujo de checkout multi-step                                     |
| <code>/client/checkout/success</code>      | <code>CheckoutSuccessPage</code>   | Confirmación de compra                                           |

Panel de Administración — `AppAdmin.tsx`

## Rutas públicas admin

| Ruta                            | Componente                       | Descripción                                        |
|---------------------------------|----------------------------------|----------------------------------------------------|
| <code>/login</code>             | <code>AdminLoginPage</code>      | Login admin (email + Google con validación de rol) |
| <code>/cotizacion/:token</code> | <code>QuotationPublicPage</code> | Cotización pública por token                       |

Rutas protegidas ( `/admin/*` )

| Ruta                                       | Componente                                  |
|--------------------------------------------|---------------------------------------------|
| <code>/admin/</code>                       | <code>AdminDashboardPage</code>             |
| <code>/admin/services</code>               | <code>AdminServicesPage</code>              |
| <code>/admin/services/:uuid</code>         | <code>AdminServiceDetailPage</code>         |
| <code>/admin/users</code>                  | <code>AdminUsersPage</code>                 |
| <code>/admin/invoices</code>               | <code>AdminInvoicesPage</code>              |
| <code>/admin/cfdi</code>                   | <code>AdminCfdiPage</code>                  |
| <code>/admin/tickets</code>                | <code>AdminTicketsPage</code>               |
| <code>/admin/game-servers</code>           | <code>AdminGameServersPage</code>           |
| <code>/admin/game-versions</code>          | <code>AdminGameVersionsPage</code>          |
| <code>/admin/quotations</code>             | <code>AdminQuotationsPage</code>            |
| <code>/admin/quotations/:uuid</code>       | <code>AdminQuotationDetailPage</code>       |
| <code>/admin/blog</code>                   | <code>AdminBlogPage</code>                  |
| <code>/admin/blog/editor</code>            | <code>AdminBlogEditorPage</code>            |
| <code>/admin/blog/categories</code>        | <code>AdminBlogCategoriesPage</code>        |
| <code>/admin/categories</code>             | <code>AdminCategoriesPage</code>            |
| <code>/admin/service-plans</code>          | <code>AdminServicePlansPage</code>          |
| <code>/admin/add-ons</code>                | <code>AdminAddOnsPage</code>                |
| <code>/admin/notifications</code>          | <code>AdminNotificationsPage</code>         |
| <code>/admin/profile</code>                | <code>AdminProfilePage</code>               |
| <code>/admin/system-status</code>          | <code>AdminSystemStatusPage</code>          |
| <code>/admin/documentation</code>          | <code>AdminDocumentationPage</code>         |
| <code>/admin/api-docs</code>               | <code>AdminApiDocumentationPage</code>      |
| <code>/admin/documentation-requests</code> | <code>AdminDocumentationRequestsPage</code> |
| <code>/admin/user-requests</code>          | <code>AdminUserRequestsPage</code>          |

## Infraestructura HTTP — apiClient

Archivo: `src/infrastructure/api/apiClient.ts`

```
// Dos instancias Axios:
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL, // ej. https://api.roke.mx/api
  withCredentials: true,                      // Envía cookies en cada request
  xsrfCookieName: 'XSRF-TOKEN',              // CSRF automático
  xsrfHeaderName: 'X-XSRF-TOKEN',
});

const rootClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL,      // Para broadcasting auth
  withCredentials: true,
});
```

Interceptor de respuesta:

- `401` → redirige a `/login?reason=session_expired`
- `419` → recarga la página (CSRF mismatch)
- Flag `_skipAuthRedirect: true` en la config del request para evitar redirect en rutas de auth

## Estado global — React Query

Configuración de caché ( `src/shared/constants/queryConfig.ts` )

| Perfil                 | <code>staleTime</code> | <code>gcTime</code> | <code>refetchOnWindowFocus</code> | <code>refetchInterval</code> | Uso típico                        |
|------------------------|------------------------|---------------------|-----------------------------------|------------------------------|-----------------------------------|
| <code>static</code>    | 15 min                 | 1 hora              | No                                | No                           | Categorías, ciclos de facturación |
| <code>dynamic</code>   | 1 min                  | 5 min               | Sí                                | 5 min                        | Dashboard stats, actividad        |
| <code>sensitive</code> | 2 min                  | 10 min              | No                                | No                           | Perfil, seguridad, 2FA            |
| <code>session</code>   | 1 min                  | 5 min               | Sí                                | No                           | Sesiones activas                  |
| Default global         | 5 min                  | 15 min              | No                                | No                           | General                           |

Lógica de retry personalizada:

- No reintenta en `401` (sesión inválida)
- Hasta 3 reintentos en errores `5xx`
- Hasta 2 reintentos en otros errores

## Naming convention de queryKeys

```
// Formato: ['entidad', 'subtipoOperación', ... parámetros]
['auth', 'me']
['gameServer', 'usage', serviceUuid]
['gameServer', 'configuration', serviceUuid]
['nest', 'eggs', nestID] // nestID en la key para no mezclar cachés
['admin', 'services', filters]
['invoices', filters]
['ticket', uuid]
```

## Contexto de autenticación

`src/application/context/AuthContext.tsx`:

- Envuelve `useCurrentUser` (query `['auth','me']`)
- Expone `{ user, isAuthenticated, isLoading, isAdmin }`
- Consumido por las rutas protegidas y los componentes de header

## WebSockets en tiempo real

### echoService ( `src/infrastructure/services/echoService.ts` )

```
// Singleton de Laravel Echo
const echo = new Echo({
  broadcaster: 'reverb',
  key: import.meta.env.VITE_REVERB_APP_KEY,
  wsHost: import.meta.env.VITE_REVERB_HOST,
  wsPort: import.meta.env.VITE_REVERB_PORT,
  wssPort: import.meta.env.VITE_REVERB_PORT,
  forceTLS: import.meta.env.VITE_REVERB_SCHEME === 'wss',
  enabledTransports: ['ws', 'wss'],
  authorizer: (channel) => ({
    authorize: (socketId, callback) => {
      rootClient.post('/broadcasting/auth', { socket_id: socketId, channel_name: channel.name })
        .then(res => callback(false, res.data))
        .catch(err => callback(true, err));
    },
  }),
});
```

## Canales utilizados

| Canal                               | Tipo    | Datos                                 |
|-------------------------------------|---------|---------------------------------------|
| <code>private-user.{userId}</code>  | Privado | Notificaciones personales del cliente |
| <code>private-admin.{userId}</code> | Privado | Notificaciones del panel admin        |
| <code>private-chat.{roomId}</code>  | Privado | Mensajes de chat de soporte           |

## Consola de game server

El frontend conecta **directamente** a Pterodactyl Wings (no pasa por Reverb):



1. `GET /services/{uuid}/game-server/websocket` → devuelve `{ socket, token }` de Wings
2. Frontend abre WebSocket nativo a la URL de Wings con el token
3. Protocolo Wings: autenticar → `{ event: "auth", args: [token] }` → luego recibir eventos `console output`, `stats`, `status`

## Formularios — react-hook-form + Zod

Patrón estándar en formularios:

```
const schema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
});

const { register, handleSubmit, formState: { errors } } = useForm({
  resolver: zodResolver(schema),
});

const mutation = useLogin();

const onSubmit = (data) => mutation.mutate(data);
```

## Internacionalización (i18n)

- Configurado con `i18next` + `react-i18next`
- Archivos de traducción en `src/shared/utils/i18n/`
- Idioma por defecto: español ( `es` )
- Uso: `const { t } = useTranslation(); t('key.nested')`

## Monitoreo con Sentry

- Inicializado en el entry point con `Sentry.init({ dsn: VITE_SENTRY_DSN })`
- `ErrorBoundary` de Sentry envuelve la app completa
- Errores no manejados, promesas rechazadas y errores de React capturados automáticamente
- `SectionErrorBoundary` para secciones individuales con fallback UI

## Arquitectura de componentes de UI

### Jerarquía

```

Page (lógica de datos, layout general)
├─ Layout Component (header, sidebar, tabs)
│   └─ Feature Component (lógica de presentación + acciones)
│       └─ UI Component (shadcn/ui primitivos: Button, Dialog, Badge...)

```

### Primitivos UI (shadcn/ui + Radix)

Disponibles en `src/presentation/components/ui/`:

[Button](#), [Input](#), [Select](#), [Checkbox](#), [Badge](#), [Dialog](#), [Sheet](#), [Tabs](#), [Card](#), [Table](#), [Pagination](#), [Avatar](#), [Dropdown](#),  
[Tooltip](#), [Toast](#), [Skeleton](#), [Switch](#), [Textarea](#), [Label](#), [Separator](#), [ScrollArea](#), [Collapsible](#), [Command](#), [Popover](#),  
[Calendar](#), [DatePicker](#), ...

## Testing

### Tests unitarios y de integración (Vitest + Testing Library)

```

pnpm test           # Modo watch
pnpm test --run     # Una sola pasada
pnpm test:coverage  # Con cobertura

```

### Mocking de API (MSW)

`src/mocks/` contiene handlers para interceptar requests HTTP en tests:

```

// Ejemplo de handler
http.get('/api/auth/me', () => HttpResponse.json({ data: mockUser }))

```

### Tests E2E (Playwright)

```

pnpm test:e2e      # Ejecutar tests E2E
pnpm test:e2e --ui  # Interfaz visual de Playwright

```

## Patrón de desarrollo de nuevas funcionalidades

1. Definir el tipo en `src/core/` o inline en el servicio
2. Crear el servicio en `src/infrastructure/services/`
3. Crear el hook en `src/application/hooks/` (React Query)
4. Crear el componente en `src/presentation/components/features/`
5. Crear la página en `src/presentation/pages/`

6. Registrar la ruta en `AppPortal.tsx` o `AppAdmin.tsx`

## Referencia de Hooks — Hosting Platform Frontend

Todos los hooks viven en `src/application/hooks/`.

Usan TanStack React Query v5 internamente.

Los hooks de **Query** devuelven `{ data, isLoading, isError, error, refetch }`.

Los hooks de **Mutation** devuelven `{ mutate, mutateAsync, isPending, isError, error, isSuccess }`.

### Autenticación ( `useAuth.ts` )

| Hook                                   | Tipo     | Descripción                                                                                                              | Retorna                                       |
|----------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| <code>useCurrentUser()</code>          | Query    | Usuario autenticado actual. <code>queryKey: ['auth', 'me']</code> , <code>staleTime: 5min</code> , sin refetch en focus. | <code>User</code>   <code>undefined</code>    |
| <code>useLogin()</code>                | Mutation | <code>POST /auth/login</code> . Invalida <code>['auth', 'me']</code> en éxito.                                           | —                                             |
| <code>useLoginWithGoogle()</code>      | Mutation | <code>POST /auth/google/callback</code> con <code>credential: id_token</code> .                                          | <code>{ requires_setup, setup_token? }</code> |
| <code>useAdminLoginWithGoogle()</code> | Mutation | <code>POST /admin/auth/google/callback</code> . Valida rol admin.                                                        | —                                             |
| <code>useRegister()</code>             | Mutation | <code>POST /auth/register</code> .                                                                                       | —                                             |
| <code>useVerify2FA()</code>            | Mutation | <code>POST /auth/2fa/verify</code> con <code>{ code, temp_token }</code> .                                               | —                                             |
| <code>useLogout()</code>               | Mutation | <code>POST /auth/logout</code> . Limpia todo el query cache.                                                             | —                                             |
| <code>useCompleteProfile()</code>      | Mutation | <code>POST /auth/complete-profile</code> con <code>{ setup_token, username }</code> .                                    | —                                             |
| <code>useSetupUsername()</code>        | Mutation | <code>POST /auth/setup-username</code> . Para usuarios autenticados sin username.                                        | —                                             |
| <code>useForgotPassword()</code>       | Mutation | <code>POST /forgot-password</code> .                                                                                     | —                                             |
| <code>useResetPassword()</code>        | Mutation | <code>POST /reset-password</code> con token.                                                                             | —                                             |

### Dashboard cliente ( `useDashboard.ts` )

| Hook                                | Tipo  | queryKey                               | Descripción                                                                                                                                 |
|-------------------------------------|-------|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>useDashboardStats()</code>    | Query | <code>['dashboard', 'stats']</code>    | Métricas: servicios activos, gasto mensual, tickets abiertos, dominios. <code>staleTime: 1min</code> , <code>refetchInterval: 5min</code> . |
| <code>useDashboardServices()</code> | Query | <code>['dashboard', 'services']</code> | Servicios del cliente para el dashboard.                                                                                                    |
| <code>useDashboardActivity()</code> | Query | <code>['dashboard', 'activity']</code> | Actividad reciente. <code>refetchOnWindowFocus: true</code> .                                                                               |

## Servicios cliente ( `useServices.ts` )

| Hook                                      | Tipo     | queryKey                                   | Descripción                                                                                      |
|-------------------------------------------|----------|--------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>useUserServices()</code>            | Query    | <code>['services', 'user']</code>          | Todos los servicios del usuario. Normaliza specs, metrics, billing_cycle.                        |
| <code>useServiceDetails(uuid)</code>      | Query    | <code>['service', uuid]</code>             | Detalle completo de un servicio.                                                                 |
| <code>useServiceInvoices(uuid)</code>     | Query    | <code>['service', uuid, 'invoices']</code> | Facturas asociadas al servicio.                                                                  |
| <code>useUpdateServiceConfig(uuid)</code> | Mutation | —                                          | <code>PATCH /services/{uuid}/configuration</code> .<br>Invalida <code>['service', uuid]</code> . |
| <code>useServiceBackups(uuid)</code>      | Query    | <code>['service', uuid, 'backups']</code>  | Lista de backups.                                                                                |
| <code>useCreateBackup(uuid)</code>        | Mutation | —                                          | <code>POST /services/{uuid}/backups</code> . Invalida lista de backups.                          |
| <code>useRestoreBackup(uuid)</code>       | Mutation | —                                          | <code>POST /services/{uuid}/backups/{backupId}/restore</code> .                                  |
| <code>useServicesMetrics()</code>         | Query    | <code>['services', 'metrics']</code>       | Métricas de todos los servicios del usuario.                                                     |

Game Server cliente ( `useGameServer.ts` )

| Hook                                                      | Tipo     | queryKey                                             | Descripción                                                                                                                                                         |
|-----------------------------------------------------------|----------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>useGameServerUsage(uuid, enabled?)</code>           | Query    | <code>['gameServer', 'usage', uuid]</code>           | Métricas en tiempo real (CPU, RAM, red, disco).<br><code>refetchInterval: 30s</code> ,<br><code>staleTime: 0</code> .                                               |
| <code>useGameServerSoftwareOptions(uuid, enabled?)</code> | Query    | <code>['gameServer', 'softwareOptions', uuid]</code> | Opciones de software disponibles (eggs del plan).<br><code>staleTime: 10min</code> .                                                                                |
| <code>useGameServerConfiguration(uuid, enabled?)</code>   | Query    | <code>['gameServer', 'configuration', uuid]</code>   | Configuración actual: software, server.properties, EULA, Java.<br><code>staleTime: 30s</code> .                                                                     |
| <code>useUpdateGameServerSoftware(uuid)</code>            | Mutation | —                                                    | <code>PATCH</code><br><code>/services/{uuid}/game-server/software</code> . Invalida <code>configuration</code> y <code>service</code> .                             |
| <code>useUpdateGameServerProperties(uuid)</code>          | Mutation | —                                                    | <code>PATCH</code><br><code>/services/{uuid}/game-server/server-properties</code> .                                                                                 |
| <code>useAcceptGameServerEula(uuid)</code>                | Mutation | —                                                    | <code>POST</code><br><code>/services/{uuid}/game-server/eula/accept</code> . Invalida <code>configuration</code> .                                                  |
| <code>useFixGameServerJava(uuid)</code>                   | Mutation | —                                                    | <code>POST</code><br><code>/services/{uuid}/game-server/fix-java</code> con <code>targetJava?: number</code> . Invalida <code>configuration</code> .                |
| <code>useGameServerPower(uuid)</code>                     | Mutation | —                                                    | <code>POST</code><br><code>/services/{uuid}/game-server/power</code> con signal: <code>'start'   'stop'   'restart'   'kill'</code> . Invalida <code>usage</code> . |
| <code>useNestEggs(nestID, enabled?)</code>                | Query    | <code>['nest', 'eggs', nestID]</code>                | Eggs de Pterodactyl para un nido. <code>enabled</code> requiere <code>nestID &gt; 0</code> . <code>staleTime: 10min</code> .                                        |
| <code>useGameServerStartup(uuid)</code>                   | Query    | <code>['gameServer', 'startup', uuid]</code>         | Configuración de arranque del servidor (env vars, variables Pterodactyl).                                                                                           |

## Checkout ( `useCheckout.ts` )

| Hook                               | Tipo  | queryKey                            | Descripción                                                                    |
|------------------------------------|-------|-------------------------------------|--------------------------------------------------------------------------------|
| <code>useGameEggs(planUuid)</code> | Query | <code>['gameEggs', planUuid]</code> | Eggs agrupados por nido para un plan. Usado en selector de juego del checkout. |
| <code>usePlanAddons(planId)</code> | Query | <code>['planAddons', planId]</code> | Add-ons disponibles para un plan.                                              |
| <code>usePaymentMethods()</code>   | Query | <code>['paymentMethods']</code>     | Métodos de pago guardados del usuario.                                         |

## Facturas e invoices ( `useInvoices.ts` )

| Hook                                          | Tipo     | Descripción                                                                                                  |
|-----------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------|
| <code>useInvoices(filters?)</code>            | Query    | Lista paginada de facturas. Acepta filtros <code>status</code> , <code>page</code> , <code>per_page</code> . |
| <code>useInvoiceStats()</code>                | Query    | Totales, pagadas, pendientes, vencidas.                                                                      |
| <code>useProcessPayment()</code>              | Mutation | <code>POST /payments/process</code> con <code>{ invoice_id, payment_method_id }</code> .                     |
| <code>useSetDefaultPaymentMethod()</code>     | Mutation | <code>PUT /payments/methods/{id}</code> → establece como predeterminado.                                     |
| <code>useDeletePaymentMethod()</code>         | Mutation | <code>DELETE /payments/methods/{id}</code> .                                                                 |
| <code>useAddPaymentMethod()</code>            | Mutation | <code>POST /payments/methods</code> con <code>payment_method_id</code> de Stripe.                            |
| <code>useCreateSetupIntent()</code>           | Mutation | <code>POST /payments/setup-intent</code> → devuelve <code>client_secret</code> de Stripe.                    |
| <code>useUpdateInvoiceFiscalData(uuid)</code> | Mutation | <code>PUT /invoices/{uuid}/fiscal-data</code> . Solo disponible en ventana de 72h.                           |
| <code>useDownloadCfdi(uuid, format)</code>    | Mutation | <code>GET /invoices/{uuid}/pdf</code> o <code>/xml</code> . Descarga automática del blob.                    |
| <code>useDownloadReceipt(uuid)</code>         | Mutation | <code>GET /invoices/{uuid}/receipt</code> . Descarga blob.                                                   |
| <code>useAdminDownloadReceipt(uuid)</code>    | Mutation | <code>GET /admin/invoices/{uuid}/receipt</code> . Versión admin.                                             |
| <code>useTransactions(filters?)</code>        | Query    | Historial de transacciones con filtros.                                                                      |
| <code>useTransactionStats()</code>            | Query    | Estadísticas de transacciones.                                                                               |

## Tickets de soporte ( `useTickets.ts` )

| Hook                              | Tipo     | Descripción                                                                             |
|-----------------------------------|----------|-----------------------------------------------------------------------------------------|
| <code>useTickets(params?)</code>  | Query    | Lista de tickets del usuario con filtros ( <code>status</code> , <code>search</code> ). |
| <code>useTicket(uuid)</code>      | Query    | Detalle de ticket.                                                                      |
| <code>useTicketStats()</code>     | Query    | Estadísticas: abiertos, resueltos, en espera.                                           |
| <code>useCreateTicket()</code>    | Mutation | <code>POST /tickets</code> . Invalida lista.                                            |
| <code>useAddReply(uuid)</code>    | Mutation | <code>POST /tickets/{uuid}/reply</code> . Admite <code>FormData</code> para adjuntos.   |
| <code>useCloseTicket(uuid)</code> | Mutation | <code>PUT /tickets/{uuid}/close</code> .                                                |

## Perfil cliente ( `useProfile.ts` )

| Hook                                      | Tipo           | Descripción                                                                                                     |
|-------------------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------|
| <code>useProfile()</code>                 | Query          | Datos del perfil del usuario. Con optimistic update.                                                            |
| <code>useUpdateProfile()</code>           | Mutation       | <code>PUT /profile</code> . Optimistic update + rollback en error.                                              |
| <code>useUploadAvatar()</code>            | Mutation       | <code>POST /profile/avatar</code> (FormData). Invalida <code>['profile']</code> y <code>['auth', 'me']</code> . |
| <code>useSecurity()</code>                | Query          | Resumen de seguridad (2FA activo, sesiones).                                                                    |
| <code>useUpdatePassword()</code>          | Mutation       | <code>PUT /profile/password</code> .                                                                            |
| <code>useResendEmailVerification()</code> | Mutation       | <code>POST /profile/email/verification-notification</code> .                                                    |
| <code>useSessions()</code>                | Query          | Dispositivos activos. <code>refetchOnWindowFocus: true</code> .                                                 |
| <code>useRevokeSession(uuid)</code>       | Mutation       | <code>DELETE /profile/sessions/{uuid}</code> .                                                                  |
| <code>useRevokeOtherSessions()</code>     | Mutation       | <code>POST /profile/devices/revoke-others</code> .                                                              |
| <code>useTwoFactor()</code>               | Hook compuesto | CRUD de 2FA: <code>generate</code> , <code>enable</code> , <code>disable</code> , <code>verify</code> .         |

## Fiscal / CFDI cliente ( `useFiscal.ts` )

| Hook                                          | Tipo     | Descripción                                                   |
|-----------------------------------------------|----------|---------------------------------------------------------------|
| <code>useFiscalProfiles()</code>              | Query    | Perfiles fiscales guardados (RFC, razón social, régimen).     |
| <code>useCreateFiscalProfile()</code>         | Mutation | <code>POST /fiscal/profiles</code> .                          |
| <code>useUpdateFiscalProfile(uuid)</code>     | Mutation | <code>PUT /fiscal/profiles/{uuid}</code> .                    |
| <code>useDeleteFiscalProfile(uuid)</code>     | Mutation | <code>DELETE /fiscal/profiles/{uuid}</code> .                 |
| <code>useSetDefaultFiscalProfile(uuid)</code> | Mutation | <code>PUT /fiscal/profiles/{uuid}/set-default</code> .        |
| <code>useFiscalRegimes()</code>               | Query    | Catálogo de regímenes SAT. <code>staleTime: 15min</code> .    |
| <code>useCfdiUses()</code>                    | Query    | Catálogo de usos de CFDI SAT. <code>staleTime: 15min</code> . |

## Notificaciones cliente ( `useClientNotifications.ts` )

| Hook                                            | Tipo     | Descripción                                              |
|-------------------------------------------------|----------|----------------------------------------------------------|
| <code>useClientNotifications()</code>           | Query    | Lista de notificaciones del usuario.                     |
| <code>useUnreadNotificationsCount()</code>      | Query    | Conteo de no leídas. <code>refetchInterval: 30s</code> . |
| <code>useMarkNotificationRead(id)</code>        | Mutation | <code>PUT /notifications/{id}/read</code> .              |
| <code>useMarkAllNotificationsRead()</code>      | Mutation | <code>PUT /notifications/mark-all-read</code> .          |
| <code>useDeleteNotification(id)</code>          | Mutation | <code>DELETE /notifications/{id}</code> .                |
| <code>useNotificationPreferences()</code>       | Query    | Preferencias de notificación.                            |
| <code>useUpdateNotificationPreferences()</code> | Mutation | <code>PUT /notifications/preferences</code> .            |

## Chat de soporte ( `useSupportChat.ts` )

| Hook                                      | Tipo     | Descripción                                       |
|-------------------------------------------|----------|---------------------------------------------------|
| <code>useSupportRoom()</code>             | Query    | Obtiene o crea la sala de soporte del usuario.    |
| <code>useChatMessages(ticketId)</code>    | Query    | Mensajes de una sala. Se actualiza vía WebSocket. |
| <code>useSendChatMessage(ticketId)</code> | Mutation | <code>POST /chat/{ticket}/messages</code> .       |
| <code>useChatUnreadCount()</code>         | Query    | Mensajes no leídos en chat.                       |
| <code>useMarkChatRead(ticketId)</code>    | Mutation | <code>PUT /chat/{ticket}/read</code> .            |



## Admin — Dashboard ( `useAdminDashboard.ts` )

| Hook                                      | Tipo     | Descripción                                                                |
|-------------------------------------------|----------|----------------------------------------------------------------------------|
| <code>useAdminStats()</code>              | Query    | Métricas globales: usuarios totales, servicios activos, ingresos, tickets. |
| <code>useAdminUsers(params?)</code>       | Query    | Lista de usuarios con filtros.                                             |
| <code>useCreateUser()</code>              | Mutation | <code>POST /admin/users</code> .                                           |
| <code>useUpdateUser(id)</code>            | Mutation | <code>PUT /admin/users/{id}</code> .                                       |
| <code>useDeleteUser(id)</code>            | Mutation | <code>DELETE /admin/users/{id}</code> .                                    |
| <code>useUpdateUserStatus(id)</code>      | Mutation | <code>PUT /admin/users/{id}/status</code> .                                |
| <code>useAdminServices(params?)</code>    | Query    | Lista de servicios con filtros.                                            |
| <code>useUpdateServiceStatus(uuid)</code> | Mutation | <code>PUT /admin/services/{uuid}/status</code> .                           |
| <code>useAdminInvoices(params?)</code>    | Query    | Lista de facturas.                                                         |
| <code>useCreateInvoice()</code>           | Mutation | <code>POST /admin/invoices</code> .                                        |
| <code>useUpdateInvoiceStatus(id)</code>   | Mutation | <code>PUT /admin/invoices/{id}/status</code> .                             |
| <code>useMarkInvoiceAsPaid(id)</code>     | Mutation | <code>POST /admin/invoices/{id}/mark-paid</code> .                         |
| <code>useAdminTickets(params?)</code>     | Query    | Lista de tickets.                                                          |
| <code>useUpdateTicketStatus(id)</code>    | Mutation | <code>PUT /admin/tickets/{id}/status</code> .                              |
| <code>useAssignTicket(id)</code>          | Mutation | <code>POST /admin/tickets/{id}/assign</code> .                             |
| <code>useAddTicketReply(id)</code>        | Mutation | <code>POST /admin/tickets/{id}/reply</code> .                              |
| <code>useTicketCategories()</code>        | Query    | Categorías de tickets. <code>staleTime: 15min</code> .                     |
| <code>useSupportAgents()</code>           | Query    | Agentes disponibles para asignación.                                       |

## Admin — Servicios ( `useAdminServices.ts` )

| Hook                                              | Tipo     | Descripción                                                                                                                                |
|---------------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>useAdminServices(params?)</code>            | Query    | Lista paginada con filtros: <code>search</code> , <code>status</code> , <code>plan_id</code> , <code>page</code> , <code>per_page</code> . |
| <code>useAdminService(uuid)</code>                | Query    | Detalle de un servicio.                                                                                                                    |
| <code>useAdminServiceSupportOverview(uuid)</code> | Query    | Tickets y soporte relacionados al servicio.                                                                                                |
| <code>useAdminServicesStats()</code>              | Query    | Estadísticas de servicios.                                                                                                                 |
| <code>useAdminServiceHistory(uuid)</code>         | Query    | Historial de cambios del servicio.                                                                                                         |
| <code>useAdminServicesByUser(userId)</code>       | Query    | Servicios de un usuario específico.                                                                                                        |
| <code>useCreateAdminService()</code>              | Mutation | <code>POST /admin/services</code> .                                                                                                        |
| <code>useUpdateAdminService(uuid)</code>          | Mutation | <code>PUT /admin/services/{uuid}</code> .                                                                                                  |
| <code>useDeleteAdminService(uuid)</code>          | Mutation | <code>DELETE /admin/services/{uuid}</code> .                                                                                               |
| <code>useChangeAdminServiceStatus(uuid)</code>    | Mutation | <code>PUT /admin/services/{uuid}/status</code> .                                                                                           |
| <code>useSuspendAdminService(uuid)</code>         | Mutation | Cambia estado a <code>suspended</code> .                                                                                                   |
| <code>useReactivateAdminService(uuid)</code>      | Mutation | Cambia estado a <code>active</code> .                                                                                                      |

## Admin — Facturas ( `useAdminInvoices.ts` )

| Hook                                    | Tipo     | Descripción                                            |
|-----------------------------------------|----------|--------------------------------------------------------|
| <code>useAdminInvoices(params?)</code>  | Query    | Lista paginada de facturas.                            |
| <code>useAdminInvoice(id)</code>        | Query    | Detalle de factura.                                    |
| <code>useAdminInvoicesStats()</code>    | Query    | Estadísticas de facturación.                           |
| <code>useAdminOverdueInvoices()</code>  | Query    | Facturas vencidas.                                     |
| <code>useAdminRevenueReport()</code>    | Query    | Reporte de ingresos.                                   |
| <code>useCreateAdminInvoice()</code>    | Mutation | Crear factura manual.                                  |
| <code>useUpdateAdminInvoice(id)</code>  | Mutation | Actualizar factura.                                    |
| <code>useDeleteAdminInvoice(id)</code>  | Mutation | Eliminar factura.                                      |
| <code>useMarkInvoiceAsPaid(id)</code>   | Mutation | <code>POST /admin/invoices/{id}/mark-paid</code> .     |
| <code>useUpdateInvoiceStatus(id)</code> | Mutation | Cambiar estado.                                        |
| <code>useCancelInvoice(id)</code>       | Mutation | <code>POST /admin/invoices/{id}/cancel</code> .        |
| <code>useSendInvoiceReminder(id)</code> | Mutation | <code>POST /admin/invoices/{id}/send-reminder</code> . |

## Admin — Tickets ( `useAdminTickets.ts` )

| Hook                                     | Tipo     | Descripción                                                   |
|------------------------------------------|----------|---------------------------------------------------------------|
| <code>useAdminTickets(params?)</code>    | Query    | Lista de tickets con filtros avanzados.                       |
| <code>useAdminTicket(id)</code>          | Query    | Detalle de ticket.                                            |
| <code>useAdminTicketsStats()</code>      | Query    | Estadísticas: abiertos, resueltos, por prioridad, por agente. |
| <code>useAdminTicketCategories()</code>  | Query    | Categorías disponibles.                                       |
| <code>useAdminTicketAgents()</code>      | Query    | Agentes con sus estadísticas.                                 |
| <code>useAdminTicketReplies(id)</code>   | Query    | Respuestas de un ticket.                                      |
| <code>useAdminTicketPerformance()</code> | Query    | Métricas de rendimiento de agentes.                           |
| <code>useCreateAdminTicket()</code>      | Mutation | Crear ticket desde admin.                                     |
| <code>useUpdateAdminTicket(id)</code>    | Mutation | Actualizar ticket.                                            |
| <code>useDeleteAdminTicket(id)</code>    | Mutation | Eliminar ticket.                                              |
| <code>useChangeTicketStatus(id)</code>   | Mutation | Cambiar estado.                                               |
| <code>useChangeTicketPriority(id)</code> | Mutation | Cambiar prioridad.                                            |
| <code>useAssignTicket(id)</code>         | Mutation | Asignar a agente.                                             |
| <code>useAddTicketReply(id)</code>       | Mutation | Agregar respuesta.                                            |
| <code>useCloseTicket(id)</code>          | Mutation | Cerrar ticket.                                                |
| <code>useReopenTicket(id)</code>         | Mutation | Reabrir ticket.                                               |

## Admin — Usuarios ( `useUsers.ts` )

| Hook                                  | Tipo     | Descripción                                                                                                                                |
|---------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>useUsers(params?)</code>        | Query    | Lista de usuarios con filtros: <code>search</code> , <code>status</code> , <code>role</code> , <code>page</code> , <code>per_page</code> . |
| <code>useUser(id)</code>              | Query    | Detalle de usuario.                                                                                                                        |
| <code>useCreateUser()</code>          | Mutation | <code>POST /admin/users</code> .                                                                                                           |
| <code>useUpdateUser(id)</code>        | Mutation | <code>PUT /admin/users/{id}</code> .                                                                                                       |
| <code>useDeleteUser(id)</code>        | Mutation | <code>DELETE /admin/users/{id}</code> .                                                                                                    |
| <code>useChangeUserStatus(id)</code>  | Mutation | <code>PUT /admin/users/{id}/status</code> .                                                                                                |
| <code>useUsersStats()</code>          | Query    | Estadísticas de usuarios (totales, activos, nuevos esta semana).                                                                           |
| <code>useUsersRecentActivity()</code> | Query    | Actividad reciente de usuarios.                                                                                                            |

## Admin — Game Servers ( `useAdminGameServers.ts` )

| Hook                                      | Tipo     | Descripción                                                                       |
|-------------------------------------------|----------|-----------------------------------------------------------------------------------|
| <code>useAdminGameServers(params?)</code> | Query    | Lista de game servers con filtros.                                                |
| <code>useAdminGameServer(id)</code>       | Query    | Detalle de game server.                                                           |
| <code>useProvisionGameServer(id)</code>   | Mutation | <code>POST /admin/game-servers/{id}/provision</code> . Provisiona en Pterodactyl. |
| <code>useSuspendGameServer(id)</code>     | Mutation | <code>POST /admin/game-servers/{id}/suspend</code> .                              |
| <code>useUnsuspendGameServer(id)</code>   | Mutation | <code>POST /admin/game-servers/{id}/unsuspend</code> .                            |
| <code>useReinstallGameServer(id)</code>   | Mutation | <code>POST /admin/game-servers/{id}/reinstall</code> .                            |
| <code>useDeleteGameServer(id)</code>      | Mutation | <code>DELETE /admin/game-servers/{id}</code> .                                    |

## Admin — CFDI ( `useAdminCfdi.ts` )

| Hook                                          | Tipo     | Descripción                                                                                       |
|-----------------------------------------------|----------|---------------------------------------------------------------------------------------------------|
| <code>useAdminCfdi(params?)</code>            | Query    | Lista de CFDIs con filtros: <code>status</code> , <code>date_from</code> , <code>date_to</code> . |
| <code>useAdminCfdiStats()</code>              | Query    | Estadísticas: timbrados, fallidos, cancelados, monto total.                                       |
| <code>useAdminCfdiDetail(id)</code>           | Query    | Detalle de un CFDI.                                                                               |
| <code>useRetryCfdi(id)</code>                 | Mutation | <code>POST /admin/cfdi/{id}/retry</code> . Reintenta timbrado.                                    |
| <code>useCancelCfdi(id)</code>                | Mutation | <code>POST /admin/cfdi/{id}/cancel</code> . Cancela ante SAT.                                     |
| <code>useDownloadAdminCfdi(id, format)</code> | Mutation | <code>GET /admin/cfdi/{id}/download/{format}</code> . Descarga PDF o XML.                         |

## Admin — Cotizaciones ( `useQuotations.ts` )

| Hook                                          | Tipo     | Descripción                              |
|-----------------------------------------------|----------|------------------------------------------|
| <code>useQuotations(params?)</code>           | Query    | Lista de cotizaciones con filtros.       |
| <code>useQuotation(uuid)</code>               | Query    | Detalle de cotización.                   |
| <code>usePublicQuotation(token)</code>        | Query    | Cotización pública por token (sin auth). |
| <code>useCreateQuotation()</code>             | Mutation | Crear cotización.                        |
| <code>useUpdateQuotation(uuid)</code>         | Mutation | Actualizar.                              |
| <code>useDeleteQuotation(uuid)</code>         | Mutation | Eliminar.                                |
| <code>useSendQuotation(uuid)</code>           | Mutation | Enviar al cliente por email.             |
| <code>useRegenerateQuotationLink(uuid)</code> | Mutation | Generar nuevo token de acceso público.   |
| <code>useAcceptQuotation(uuid)</code>         | Mutation | Marcar como aceptada.                    |
| <code>useRejectQuotation(uuid)</code>         | Mutation | Marcar como rechazada.                   |
| <code>useReopenQuotation(uuid)</code>         | Mutation | Reabrir cotización expirada/rechazada.   |
| <code>useCreateQuotationRevision(uuid)</code> | Mutation | Crear nueva versión de la cotización.    |

## Admin — Game Versions ( `useAdminGameVersions.ts` )

| Hook                                        | Tipo     | Descripción                                                                                                                                 |
|---------------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>useAdminGameVersions(filters?)</code> | Query    | Lista de versiones de software ( <code>software</code> , <code>enabled</code> , <code>page</code> ).                                        |
| <code>useCreateAdminGameVersion()</code>    | Mutation | Crear versión.                                                                                                                              |
| <code>useUpdateAdminGameVersion(id)</code>  | Mutation | Actualizar versión (nombre, URL de descarga, estado recomendado).                                                                           |
| <code>useDeleteAdminGameVersion(id)</code>  | Mutation | Eliminar versión.                                                                                                                           |
| <code>useBulkAdminGameVersions()</code>     | Mutation | <code>POST /admin/game-versions/bulk/{action}</code> . Acciones en lote: <code>enable</code> , <code>disable</code> , <code>delete</code> . |

## Búsqueda global ( `useGlobalSearch.ts` )

```
const { query, results, isLoading, search, clearSearch, hasResults, popularQueries, recentSearches } = useGlobalSearch();
```

- Query debounced **400ms**.
- Mínimo **2 caracteres** para disparar búsqueda.
- `recentSearches` almacenado en **localStorage**.
- `popularQueries` via `GET /admin/search/popular` .

## Catálogo admin ( `useServicePlans.ts` , `useAdminAddOns.ts` )

| Hook                                       | Descripción                        |
|--------------------------------------------|------------------------------------|
| <code>useAdminServicePlans(params?)</code> | Lista de planes con filtros.       |
| <code>useCreateServicePlan()</code>        | Crear plan.                        |
| <code>useUpdateServicePlan(uuid)</code>    | Actualizar plan.                   |
| <code>useDeleteServicePlan(uuid)</code>    | Eliminar plan.                     |
| <code>useBulkServicePlans()</code>         | Acciones en lote.                  |
| <code>useAdminCategories()</code>          | Categorías de servicios.           |
| <code>useAdminAddOns(params?)</code>       | Lista de add-ons.                  |
| <code>useCreateAdminAddOn()</code>         | Crear add-on.                      |
| <code>useUpdateAdminAddOn(uuid)</code>     | Actualizar add-on.                 |
| <code>useDeleteAdminAddOn(uuid)</code>     | Eliminar add-on.                   |
| <code>useAttachAddonToPlan(uuid)</code>    | Asociar add-on a plan.             |
| <code>useDetachAddonFromPlan(uuid)</code>  | Desasociar.                        |
| <code>useBillingCycles()</code>            | Ciclos de facturación disponibles. |

## Notificaciones admin ( `useAdminNotifications.ts` )

| Hook                                            | Descripción                               |
|-------------------------------------------------|-------------------------------------------|
| <code>useAdminNotifications()</code>            | Lista de notificaciones del admin.        |
| <code>useAdminNotificationStats()</code>        | Estadísticas (no leídas, archivadas).     |
| <code>useBroadcastNotification()</code>         | Enviar notificación a todos los usuarios. |
| <code>useSendNotificationToUser(userId)</code>  | Enviar a usuario específico.              |
| <code>useMarkAdminNotificationRead(id)</code>   | Marcar como leída.                        |
| <code>useMarkAllAdminNotificationsRead()</code> | Marcar todas como leídas.                 |
| <code>useArchiveNotification(id)</code>         | Archivar.                                 |
| <code>useDeleteAdminNotification(id)</code>     | Eliminar.                                 |

## Hooks de utilidades

| Hook                                         | Ubicación                           | Descripción                                                                         |
|----------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------|
| <code>use-mobile</code>                      | <code>src/application/hooks/</code> | Breakpoint responsive. Retorna <code>isMobile: boolean</code> .                     |
| <code>useSessionManager()</code>             | <code>useSessionManager.ts</code>   | Detecta inactividad y dispara logout automático.                                    |
| <code>useHosting(uuid)</code>                | <code>useHosting.ts</code>          | Info, bases de datos, emails y dominios del hosting HestiaCP.                       |
| <code>useSoftwareVersions(identifier)</code> | <code>useSoftwareVersions.ts</code> | Versiones de software para un egg (ej. <code>vanilla</code> , <code>paper</code> ). |
| <code>useFileManager(uuid)</code>            | <code>useFileManager.ts</code>      | Listar, subir, eliminar y descargar archivos de un servicio.                        |
| <code>useAdminFileManager(id)</code>         | <code>useAdminFileManager.ts</code> | Gestor de archivos con bypass admin.                                                |
| <code>useAdminChat(roomId)</code>            | <code>useAdminChat.ts</code>        | Chat de soporte admin: mensajes, envío, asignación.                                 |
| <code>useChatInteractions(roomId)</code>     | <code>useChatInteractions.ts</code> | Interacciones del chat (typing, leídos).                                            |

## Guía de Funcionalidades — Hosting Platform Frontend

Documentación módulo a módulo de las funcionalidades implementadas en el portal de clientes y el panel de administración.

### Portal de Clientes

#### Autenticación y registro

Archivos: `src/presentation/pages/auth/`

##### Login ( `LoginPage` )

- Formulario email + contraseña con validación Zod.
- Botón Iniciar sesión con Google vía `@react-oauth/google`.
- Si el backend retorna `requires_2fa: true` → redirige a `/verify-2fa` con `temp_token` en estado.
- Si Google retorna `requires_setup: true` → redirige a `/auth/complete-profile`.
- Mensaje de razón cuando hay `?reason=session_expired` en la URL.

##### Registro ( `RegisterPage` )

- Campos: nombre, email, contraseña, confirmación de contraseña.
- Envía `POST /auth/register`. Redirige a login tras registro exitoso.

##### Verificación 2FA ( `Verify2FAPage` )

- Formulario de 6 dígitos.
- Llama a `useVerify2FA()` con el `temp_token` recibido en el login.

##### Completar perfil Google ( `CompleteProfilePage` )

- Requerido cuando el usuario nuevo se registra con Google y no tiene username.
- Verifica disponibilidad de username en tiempo real vía `GET /auth/username/check`.
- Envía `POST /auth/complete-profile { setup_token, username }`.

## Reset de contraseña

- `ForgotPasswordPage` → solicita email.
- `ResetPasswordPage` → recibe token por URL, valida contraseña nueva.

## Dashboard del cliente ( `ClientDashboardPage` )

Hook: `useDashboardStats`, `useDashboardServices`, `useDashboardActivity`

### Métricas mostradas:

- Servicios activos
- Gasto mensual actual
- Tickets de soporte abiertos
- Dominios registrados

### Secciones:

- **Servicios recientes** — lista de últimos servicios con estado y tipo.
- **Actividad reciente** — log de acciones del usuario (pagos, tickets, cambios de configuración).
- **Acciones rápidas** — accesos directos a: Contratar servicio, Crear ticket, Ver facturas, Administrar perfil.

## Servicios del cliente

### Lista de servicios ( `ClientServicesPage` )

Hook: `useUserServices()`

- Grid de tarjetas de servicios con: nombre, estado (badge), tipo (icono), fecha de próximo pago.
- Barra de búsqueda y filtros por estado.
- Click en tarjeta → navega a detalle.

### Detalle de servicio ( `ServiceDetailPage` )

Hook: `useServiceDetails(uuid)`

- Muestra specs del plan, estado, IP de conexión, fecha de renovación.
- Si el servicio es **game server** → redirige automáticamente a `GameServerDetail`.
- Si es **hosting** → redirige a `/client/hosting/:uuid`.
- **Acciones disponibles:**
  - Crear backup
  - Cancelar servicio (con confirmación)
  - Reactivar servicio (si está suspendido)

### Configuración avanzada ( `ServiceManagementPage` )

#### Tabs:

1. **General** — nombre del servicio, notas.
2. **Security** — configuraciones de acceso.
3. **Danger Zone** — cancelar servicio permanentemente.



## Game Server — Vista completa ( `GameServerDetail` )

**Archivo:** `src/presentation/components/features/services/game-server/GameServerDetail.tsx`

**Hooks:** `useGameServerUsage` , `useGameServerConfiguration` , `useGameServerPower`

El componente central para la gestión de game servers. Se adapta al tipo de egg (no solo Minecraft).

### Tabs

| Tab                       | Contenido                                             | Visible cuando                |
|---------------------------|-------------------------------------------------------|-------------------------------|
| Consola                   | Terminal WebSocket en tiempo real, control de energía | Siempre                       |
| Configuración             | Software, server.properties, propiedades técnicas     | Siempre                       |
| Mods / Plugins / Archivos | Gestor de archivos + marketplace                      | Siempre (label varía por egg) |

Label del tab de archivos ( `getFilesTabLabel` )

- Eggs con mod loader (Forge, Fabric, NeoForge, Quilt) → **"Mods"**
- Eggs con plugin loader (Paper, Spigot, Bukkit, Purpur, Arclight, Sponge, Folia) → **"Plugins"**
- Resto → **"Archivos del servidor"**

### Modales condicionales (solo eggs de Minecraft Java)

La función `isJavaMinecraftEgg(eggName)` determina si aplican los modales. Solo se activan para Java Edition; no aplican a Bedrock, BungeeCord, Velocity, Nukkit.

| Modal                 | Trigger                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| Java version mismatch | <code>javaMismatch: true</code> en la configuración, o el parser de consola detectó versión Java incompatible |
| EULA                  | <code>eulaAccepted: false</code> en la configuración, o el parser de consola detectó <code>eula.txt</code>    |

### Consola WebSocket

- `GET /services/{uuid}/game-server/websocket` → `{ socket: "wss:// ... ", token: " ... " }`
- `new WebSocket(socket)` → `send(JSON.stringify({ event: "auth", args: [token] }))`
- Eventos recibidos: `console output` (líneas de log), `stats` (CPU/RAM en tiempo real), `status` (running/offline/starting/stopping)
- Parser de consola detecta patrones:
  - `eula.txt` → activa flag `consoleDetectedEula`
  - `Java version mismatch` / `requires Java X` → activa `consoleDetectedJava`

### Control de energía

Botones: **Iniciar**, **Detener**, **Reiniciar**, **Forzar detención (Kill)**.

Usa `useGameServerPower(uuid)` → `POST /services/{uuid}/game-server/power { signal: "start|stop|restart|kill" }`.

## Game Server — Configuración ( `GameServerSettings` )

**Archivo:** `src/presentation/components/features/services/game-server/GameServerSettings.tsx`

## Tabs de configuración

| Tab               | Descripción                                                | Visible cuando                                      |
|-------------------|------------------------------------------------------------|-----------------------------------------------------|
| Software          | Selector de versión de software (vanilla, paper, forge...) | Siempre                                             |
| server.properties | Editor de propiedades del servidor                         | Solo eggs Minecraft ( <code>isMinecraftEgg</code> ) |
| Propiedades       | Variables de startup de Pterodactyl                        | Siempre                                             |

## Detección de tipo de egg

```

isJavaSoftware(name)    → EULA aplica (excluye Bedrock/proxies aunque tengan "vanilla")
isMinecraftEgg(name)    → Tab server.properties visible
isModrinthSupported(name) → Tab Marketplace visible (solo loaders: forge, fabric, paper...)

```

## Exclusiones de Java

```
JAVA_EXCLUSION_KEYWORDS = ["bedrock", "nukkit", "bungeecord", "bungee", "velocity", "proxy"]
```

Estos términos tienen prioridad sobre cualquier keyword del allowlist (ej. "Vanilla Bedrock" no activa el modal de EULA aunque "vanilla" esté en la lista Java).

## Modal EULA

- Aparece cuando `isJavaSoftware && !eulaAccepted`.
- Requiere que el usuario lea y acepte los términos de Minecraft EULA.
- Botón **Aceptar EULA** → `useAcceptGameServerEula()`.

## Modal Java version mismatch

- Aparece cuando `isJavaSoftware && javaMismatch`.
- Muestra la versión actual y la requerida.
- Botón **Corregir automáticamente** → `useFixGameServerJava()`.

## Visuales por egg ( `getSoftwareVisual` )

Cada tipo de egg tiene una combinación de **icono** + **color** + **badge** definida en `SOFTWARE_VISUAL`. Eggs soportados con visual dedicado:

Minecraft: `vanilla`, `paper`, `spigot`, `bukkit`, `fabric`, `forge`, `neoforge`, `quilt`, `arclight`, `sponge`, `folia`, `purpur`, `bungeecord`, `velocity`, `bedrock`, `nukkit`

Otros juegos: `tf2`, `insurgency`, `csgo`, `cs2`, `gmod`, `source`, `ark`, `arksa`, `rust`, `palworld`, `fivem`, `teamspeak`, `mumble`

Para eggs no listados usa un fallback genérico con `Gamepad2` como icono.

## Game Server — Mods y Archivos ( `ModsManager` )

Archivo: `src/presentation/components/features/services/game-server/ModsManager.tsx`

## Tabs del gestor

| Tab                    | Descripción                           | Visible cuando                            |
|------------------------|---------------------------------------|-------------------------------------------|
| Archivos               | Explorador de archivos del servidor   | Siempre                                   |
| Marketplace (Modrinth) | Buscador e instalador de mods/plugins | <code>isModrinthSupported(eggName)</code> |

Directorio por defecto ( `detectDirectory` )

| Tipo de egg                                            | Directorio            |
|--------------------------------------------------------|-----------------------|
| Forge, Fabric, Quilt, NeoForge                         | <code>/mods</code>    |
| Paper, Spigot, Bukkit, Arclight, Sponge, Folia, Purpur | <code>/plugins</code> |
| Resto                                                  | <code>/</code> (raíz) |

Soporte de Modrinth ( `isModrinthSupported` )

Solo eggs con mod/plugin loader soportados:

- `forge`, `fabric`, `quilt`, `neoforge` → mods
- `spigot`, `paper`, `bukkit`, `purpur`, `arclight`, `sponge`, `folia` → plugins

**No soportan Modrinth:** vanilla, bedrock, nukkit, bungeecord, velocity, cualquier servidor de otros juegos (Rust, ARK, CS2...).

Checkout multi-step ( `CheckoutPage` )

Archivo: `src/presentation/pages/client/CheckoutPage.tsx`

Hooks: `useGameEggs`, `usePlanAddons`, `usePaymentMethods`

## Flujo de pasos

```

Step 1: Selector de juego (solo si el plan es game_server)
├─ Grid de eggs agrupados por nido (Pterodactyl)
├─ Cada egg muestra: nombre, descripción, icono

Step 2: Configuración del servicio
├─ Nombre del servidor / dominio
├─ Add-ons opcionales (checkboxes con precio incremental)
├─ Ciclo de facturación (mensual, semestral, anual)
├─ Datos de facturación (RFC, razón social, régimen SAT)
├─ Soporta perfiles fiscales guardados

Step 3: Pago y confirmación
├─ Resumen del pedido con total
├─ Selector de método de pago guardado
├─ Formulario de nueva tarjeta (Stripe Elements)
├─ Botón "Confirmar y pagar"
```

## Sidebar de resumen

Siempre visible durante el checkout, muestra:

- Plan seleccionado y precio base
- Add-ons seleccionados con precios individuales
- Ciclo de facturación y descuento aplicado
- Total actualizado en tiempo real

Facturación ( `ClientInvoicesPage` )

Hooks: `useInvoices`, `useInvoiceStats`, `usePaymentMethods`, `useTransactions`

## Tabs

| Tab             | Contenido                                                                                      |
|-----------------|------------------------------------------------------------------------------------------------|
| Comprobantes    | Lista de facturas con filtros por estado (pendiente, pagada, vencida, cancelada)               |
| Métodos de pago | Tarjetas guardadas, botón agregar (Stripe Elements), botón establecer predeterminado, eliminar |
| Transacciones   | Historial de transacciones procesadas                                                          |

### Acciones por factura

- **Pagar** — abre modal de selección de método de pago → `useProcessPayment()`
- **Descargar PDF** — descarga CFDI PDF via blob
- **Descargar XML** — descarga CFDI XML via blob
- **Descargar comprobante** — comprobante de pago interno
- **Actualizar datos fiscales** — disponible en ventana de 72h tras el pago

### Flujo de pago de factura

1. Click "Pagar" → `PaymentModal` muestra métodos guardados
2. Usuario selecciona método o agrega uno nuevo
3. `POST /payments/process { invoice_id, payment_method_id }`
4. Backend procesa en Stripe y timbra CFDI si corresponde
5. Factura pasa a estado `paid`

## Tickets de soporte ( `ClientTicketsPage` )

Hooks: `useTickets`, `useCreateTicket`, `useAddReply`, `useCloseTicket`

- Lista de tickets con filtros de estado y búsqueda.
- **Crear ticket** — modal con: categoría, asunto, descripción, adjuntos.
- **Detalle de ticket** — vista de conversación con historial de respuestas.
- **Chat en tiempo real** — `TicketChatDockPortal` dockable (portal React) para chat vía WebSocket (canal `private-chat.{roomId}`).
- **Cerrar ticket** — botón con confirmación.

## Perfil ( `ClientProfilePage` )

Hooks: `useProfile`, `useSecurity`, `useTwoFactor`, `useSessions`

## Tabs

| Tab       | Contenido                                                                                                 |
|-----------|-----------------------------------------------------------------------------------------------------------|
| Perfil    | Foto (crop con react-image-crop), nombre, email, username                                                 |
| Seguridad | Cambiar contraseña, activar/desactivar 2FA (QR code), dispositivos activos                                |
| Sesiones  | Lista de sesiones activas con IP, dispositivo, fecha. Botón revocar por sesión o "cerrar todas las otras" |

### 2FA

1. `POST /2fa/generate` → devuelve secreto + imagen QR (Base64)
2. Usuario escanea con Google Authenticator / Authy

3. `POST /2fa/enable { code: "123456" }` → habilita 2FA
4. En futuros logins el backend requiere TOTP antes de emitir cookie

## Hosting HestiaCP ( `HostingDetailPage` )

Hook: `useHosting(uuid)`

- **Info general** — panel, IP, nombre de cuenta, paquete.
- **Bases de datos** — lista, crear (nombre + usuario), eliminar.
- **Cuentas de email** — lista, crear, eliminar.
- **Dominios** — dominios adicionales en la cuenta, agregar, eliminar.
- **Estadísticas** — uso de disco, ancho de banda, archivos.

## Panel de Administración

### Dashboard admin ( `AdminDashboardPage` )

Hook: `useAdminStats()`

#### Métricas principales (4 cards):

- Total de usuarios registrados
- Servicios activos
- Ingresos del mes
- Tickets abiertos

#### Gráficas:

- Ingresos mensuales (Recharts LineChart, últimos 12 meses)
- Nuevos usuarios por semana (BarChart)
- Distribución de tickets por estado (PieChart)

**Actividad reciente:** tabla de últimas acciones del sistema.

**Acciones rápidas:** links a Usuarios, Servicios, Facturas, Tickets.

### Gestión de servicios admin ( `AdminServicesPage` )

Hooks: `useAdminServices`, `useCreateAdminService`, `useUpdateAdminService`, `useDeleteAdminService`

#### Lista principal

- Tabla paginada con selector de filas (10/25/50/100/150 por página).
- Columnas: nombre, usuario, plan, categoría, estado, próximo pago, acciones.
- Filtros: búsqueda libre, estado ( `active` / `suspended` / `cancelled` / `pending` ), plan.
- Acciones por fila: ver detalle, editar, suspender/reactivar, eliminar.

#### Sheet de creación / edición

Panel lateral ( `Sheet` ) con formulario contextual según la **categoría del servicio**:

| Categoría                  | Campos técnicos adicionales                                              |
|----------------------------|--------------------------------------------------------------------------|
| <code>game_server</code>   | <b>GameServerEggPicker</b> — grid interactivo de eggs agrupados por nido |
| <code>hosting</code>       | Dominio, paquete de HestiaCP, credenciales                               |
| <code>vps</code>           | CPU, RAM, disco, nodo de Proxmox                                         |
| <code>database</code>      | Motor (MySQL/PostgreSQL), nombre, credenciales                           |
| <code>profesionales</code> | Descripción del servicio, entregables                                    |

### GameServerEggPicker

Componente interno de `AdminServicesPage`:

- Carga eggs via `useGameEggs(planUuid)` (mismo endpoint que el checkout).
- Muestra grid de eggs agrupados por nido.
- Al seleccionar: establece `egg_id` y `game` en la configuración.
- Campo de override manual para IDs de egg no listados.
- En modo edición: pre-selecciona el egg actual del servicio.
- Muestra badge con egg seleccionado.

### Gestión de usuarios admin ( `AdminUsersPage` )

Hooks: `useUsers`, `useCreateUser`, `useUpdateUser`, `useDeleteUser`, `useChangeUserStatus`

- Tabla paginada con filtros: búsqueda, estado, rol.
- **Stats cards** en la parte superior: total, activos, suspendidos, administradores.
- Sheet de creación/edición: nombre, email, rol, estado, contraseña temporal.
- Acciones: cambiar estado (activo/suspendido), ver servicios del usuario, eliminar.

### Gestión de facturas admin ( `AdminInvoicesPage` )

Hooks: `useAdminInvoices`, `useAdminInvoicesStats`, `useMarkInvoiceAsPaid`, `useCancelInvoice`

- Tabla de facturas con filtros: estado, búsqueda, rango de fechas.
- Stats: total cobrado, pendiente, vencidas, canceladas.
- Acciones: marcar pagada, enviar recordatorio, cancelar, ver detalle, descargar comprobante.

### CFDI admin ( `AdminCfdiPage` )

Hooks: `useAdminCfdi`, `useAdminCfdiStats`, `useRetryCfdi`, `useCancelCfdi`, `useDownloadAdminCfdi`

- Lista de CFDIs con estado de timbrado (timbrado, fallido, cancelado).
- Stats: total timbrados, fallidos, monto total timbrado.
- **Reintentar timbrado** — para CFDIs con estado `failed`. Vuelve a llamar a Facturama.
- **Cancelar CFDI** — cancela ante el SAT (requiere motivo de cancelación).
- **Descargar PDF/XML** — descarga blob del CFDI.

## Tickets admin ( `AdminTicketsPage` )

**Hooks:** `useAdminTickets` , `useAdminTicketsStats` , `useChangeTicketStatus` , `useChangeTicketPriority` , `useAssignTicket`

- Vista de lista con filtros avanzados: estado, prioridad, agente asignado, categoría.
- Stats: abiertos, en progreso, resueltos, promedio de resolución.
- **AdminTicketSheet** — sheet lateral con:
  - Historial completo de conversación.
  - Formulario de respuesta con adjuntos.
  - Selector de estado y prioridad.
  - Selector de agente para asignación.
  - Botones cerrar / reabrir.
  - Chat en tiempo real integrado.

## Game Servers admin ( `AdminGameServersPage` )

**Hooks:** `useAdminGameServers` , `useProvisionGameServer` , `useSuspendGameServer` , `useReinstallGameServer`

- Lista de servers con estado en Pterodactyl (online/offline/installing).
- **Provisionar** — llama al backend que crea el servidor en Pterodactyl Panel.
- **Suspender / Desuspender** — congela el servidor en Pterodactyl.
- **Reinstalar** — reinstala el egg en el servidor.
- **Eliminar** — termina y elimina el servidor de Pterodactyl.

## Versiones de software ( `AdminGameVersionsPage` )

**Hooks:** `useAdminGameVersions` , `useCreateAdminGameVersion` , `useBulkAdminGameVersions`

- Tabla de versiones con columnas: software, versión, URL de descarga, estado (habilitado/recomendado).
- Selector de filas por página (10/25/50/100/150).
- Checkboxes para selección múltiple → acciones en lote: habilitar, deshabilitar, eliminar.
- Formulario de creación/edición: software identifier, versión, URL, flags.

## Cotizaciones ( `AdminQuotationsPage` + `AdminQuotationDetailPage` )

**Hooks:** `useQuotations` , `useSendQuotation` , `useAcceptQuotation` , `useCreateQuotationRevision`

### Ciclo de vida de una cotización

```
draft → sent → viewed → accepted ✓
                        → rejected X
                        → reopen → draft (nueva revisión)
```

### Funcionalidades

- Crear cotización con líneas de items, precios, descuentos, fecha de validez.
- **Enviar** — envía email al cliente con link público.
- **Link público** ( `/cotizacion/:token` ) — el cliente puede aceptar o rechazar sin auth.
- **Marcar vista** — cuando el cliente abre el link, se registra automáticamente.
- **Revisiones** — crear nueva versión de la cotización preservando el historial.

- **Regenerar link** — invalida el token anterior y genera uno nuevo.

### Blog ( `AdminBlogPage` + `AdminBlogEditorPage` )

**Editor:** Tiptap v3 (extensiones: Bold, Italic, Heading, BulletList, Code, Image, Link, Table, Highlight, TextAlign, Placeholder, Youtube)

- **Lista de posts** con estado (borrador, publicado), fecha, categoría.
- **Editor** — rich text completo, subida de imágenes via `POST /admin/blog/upload-image`.
- **Categorías** — CRUD de categorías del blog.
- **Suscriptores** — lista de suscriptores con opción de eliminar.

### Búsqueda global admin ( `AdminHeader` + `AdminSearch` )

- Atajo de teclado `Ctrl+K` / `Cmd+K` → abre modal de búsqueda.
- Debounce de 400ms, mínimo 2 caracteres.
- Resultados agrupados por tipo: usuarios, servicios, facturas, tickets.
- Historial de búsquedas recientes en localStorage.
- Términos populares cargados desde `GET /admin/search/popular`.

### Notificaciones admin

- Bell icon en el header con badge de no leídas.
- Dropdown de notificaciones recientes.
- `AdminNotificationsPage` — centro completo: lista, marcar leída, archivar, eliminar.
- **Broadcast** — enviar notificación a todos los usuarios del sistema.
- **Enviar a usuario** — notificación personalizada a un usuario específico.

## Funcionalidades transversales

### Perfiles fiscales (CFDI 4.0)

Disponibles tanto en el checkout como en la edición de facturas:

- El usuario puede guardar múltiples perfiles fiscales (RFC, razón social, régimen fiscal SAT, uso de CFDI).
- Al seleccionar un perfil guardado, se auto-rellenan los campos.
- Validación de RFC mexicano.
- Consulta de código postal via `GET /postal-codes/{code}` → auto-rellena colonia y municipio.
- El CFDI se puede actualizar en las **72 horas** siguientes al pago.

### Notificaciones en tiempo real

- Canal `private-user.{userId}` (Reverb) → badge de notificaciones se actualiza sin polling.
- Toast automático al recibir notificación nueva.
- Tipos de notificación: pago procesado, servicio próximo a vencer, ticket respondido, CFDI emitido.



## Responsive design

- Tailwind CSS v4 con breakpoints estándar ( `sm` , `md` , `lg` , `xl` , `2xl` ).
- `use-mobile` hook detecta breakpoint `< 768px` para ajustar comportamiento de componentes.
- Sidebar admin colapsable en pantallas pequeñas.
- Tablas con scroll horizontal en móvil.